

The Magnus Model for Dynamic Pairwise Manhattan Aggregation

Exact Order Recovery, Kinetic Compilation, Shift–Meld Histograms, and Spectral Approximation

Theodore Magnus Øen  
Independent Researcher

Abstract

We study dynamic range aggregation for ordered point sequences under range translations. The update operation adds the same vector to every point in an index interval, while the query operation asks for the total pairwise Manhattan distance inside an index interval. Although the problem is stated in three dimensions, the Manhattan objective decomposes into three independent one-dimensional dynamic pairwise L_1 aggregation problems. The resulting one-dimensional problem is already nontrivial: range additions dynamically change the value order inside a query interval, so the maintained quantity is not a monoid aggregate and is not directly handled by a standard lazy segment tree.

This paper develops the *Magnus Model*, a unified exact–approximate framework for this problem. On the exact¹ side, comparison sorting gives an $O(m \log m)$ query for an interval of length m . For fixed-width integers on the word-RAM, extracting the current values from a lazy segment tree and applying radix-statistical order recovery improves this to $O(m + \log n)$ query time with $O(\log n)$ updates and linear persistent space. A static pair-space oracle instead compiles all pair-difference breakpoints using $O(n^2 \text{polylog} n)$ preprocessing and yields an epoch-based $O(n^{4/3} \text{polylog} n)$ amortized benchmark. Fragmentation-aware routing further identifies easy exact interactions through offset fragmentation, value separation, dirty mass, and memoized structure. At the center of the value-domain route is an exact hierarchical matrix factorization. For g shifted runs, let H be the run–value frequency matrix and let $K_{pq} = |v_p - v_q|$ be the ordered absolute-distance kernel. The run interaction matrix is $C = HKH^T$. Every ordered separated block of K has rank at most two, and every binary value split contributes a rank-at-most-four matrix over the runs. If η_Q is the number of active run–value-node incidences in the visited value recursion and $R(n)$ is the cost of a range count/sum primitive, the exact factors are constructed in $O(\eta_Q R(n))$ time and $O(\eta_Q)$ space. A fixed factorization with h_Q factors reduces a batch of B run-selector queries to four rectangular matrix products, in $O(\min\{B\eta_Q, \text{MM}(B, g, h_Q) + Bh_Q\})$ time. We then derive an exact lazy-incremental transform: integer value histograms are Laurent polynomials, a translation acts by monomial multiplication, the discrete curvature of the cross function is the pair-difference correlation, and a partial translation contributes the two-term delta $(z^\Delta - 1)P_S(z)$. In an exact threshold-feature space, the same update is a box convolution whose quadratic correction is a compactly supported tent kernel. These identities yield node-local lazy coordinates and a buffered Matrix-CDQ compaction bound without claiming a generic polylogarithmic exact structure. We instantiate the transform by a persistent shift–meld histogram segment tree: full-cover updates lazily shift weighted ordered maps, partial ancestors are rebuilt by work-optimal map union, and queries temporarily meld the canonical histograms. The resulting exact operation bounds depend explicitly on distinct-value supports and retain linear worst-case fallbacks. We further tighten the adaptive routes: a direct radix divide-and-aggregate evaluator computes the exact an-

¹Throughout the manuscript, “exact” means that the absolute-value kernel is not replaced by a finite-dimensional surrogate; the data structure must account for the true value ordering induced inside each query interval.

swer without materializing a sorted output; dirty correction is regrouped to remove its quadratic dirty–dirty term; compiled Cross functions admit kinetic finger evaluation under changing relative offsets; and a rent-or-buy policy compiles only interactions whose reuse pays for their breakpoints. An index-boundary entropy bound controls the size of the maximal uniform frontier. We also give a direct conditional barrier from Tripartite 3SUM: if an exact structure has preprocessing time $P(n)$, update time $U(n)$, and query time $Q(n)$, then Tripartite 3SUM on three sets of size n can be solved in $P(2n) + O(n(U(2n) + Q(2n)) + n \log n)$ time. The reduction uses only positive integers, updates one fixed half, and queries the whole array. Consequently, under the hypothesis and subquadratic preprocessing, the linear full-range radix query is conditionally optimal up to subpolynomial factors when updates are fast.

On the approximate side, we replace dynamic ordering by finite-dimensional algebra. A feature map Φ and a matrix A approximate the absolute-value kernel by $|x - y| \approx \Phi(x)^T A \Phi(y)$, while translations act linearly as $\Phi(x + \Delta) = T_\Delta \Phi(x)$. This turns CDQ cross terms into bilinear feature interactions. Fourier features give a Lazy Spectral Segment Tree with $O(D \log n)$ update and query time and $O(Dn)$ space per coordinate, under an additive kernel approximation guarantee. Because separated value blocks already have exact rank two, the refined hybrid evaluates the far field exactly and spends approximation error only on unresolved near-cusp blocks; if ρ_Q pairs are approximated on width θ , the explicit odd-cosine kernel incurs error at most $4\theta\rho_Q/(\pi^2(2D-1))$. Polynomial, Chebyshev, multiscale, and hybrid variants fit the same algebraic template.

The resulting frontier is coherent. Exact fixed-width queries admit linear radix order recovery; under the Tripartite 3SUM Hypothesis, strongly sublinear exact operations with light preprocessing are conditionally excluded; near-quadratic pair-space compilation moves the cost into preprocessing; and spectral algebra achieves polylogarithmic operations by accepting additive error. Fragmentation, pruning, and memoization are therefore acceleration mechanisms for structured workloads, not replacements for the generic exact barrier.

2012 ACM Subject Classification Theory of computation → Data structures design and analysis; Theory of computation → Computational geometry

Keywords and phrases Dynamic data structures, range addition, pairwise Manhattan distance, L_1 aggregation, integer sorting, radix aggregation, hierarchical matrices, exact low-rank factorization, Laurent histograms, difference correlation, shift–meld histograms, kinetic cross functions, rent-or-buy compilation, dirty regrouping, direct radix aggregation, lazy incremental updates, 3SUM hardness, conditional lower bounds, value-domain divide and conquer, spectral approximation, range searching

Digital Object Identifier Not assigned

Category Technical analysis and research draft

Funding No external funding



© Theodore Magnus Øen;
licensed under [Creative Commons License CC-BY 4.0](https://creativecommons.org/licenses/by/4.0/)
Technical Manuscript.
Editors: Theodore Magnus Øen;
Article No. 0; pp. 0:1–0:36

TECHNICAL ANALYSIS

Independent research manuscript in algorithms and data structures
Self-contained proceedings-style technical draft.

1 Introduction

Dynamic range data structures are a basic tool in algorithms. In their most familiar form, they maintain an aggregate such as a sum, minimum, maximum, or affine statistic under updates on intervals. The success of Fenwick trees, segment trees, and lazy propagation comes from the fact that such aggregates are composable: the value of a parent interval can be recovered from a constant-size summary of its children. Many geometric and statistical quantities, however, are not composable in this way. A particularly simple example is the total pairwise distance inside a range.

This paper considers the following dynamic problem. We maintain an ordered sequence of points

$$P_i = (x_i, y_i, z_i) \in \mathbb{R}^3, \quad 1 \leq i \leq n.$$

An update translates all points in an index interval by a common vector, and a query asks for the sum of pairwise Manhattan distances inside an index interval. In other words, the update is

$$P_i \leftarrow P_i + (\Delta_x, \Delta_y, \Delta_z) \quad (i \in [L, R]),$$

and the query asks for

$$\text{Ans}(l, r) = \sum_{l \leq i < j \leq r} (|x_i - x_j| + |y_i - y_j| + |z_i - z_j|).$$

Since the Manhattan metric separates by coordinate, the main difficulty is already present in the one-dimensional subproblem

$$F(l, r) = \sum_{l \leq i < j \leq r} |a_i - a_j|.$$

At first glance this resembles an ordinary range-query problem, but the resemblance is deceptive. The quantity $F(l, r)$ depends on the order of the values inside the query interval, not merely on their sum or on any constant-size algebraic summary. A range addition shifts many values simultaneously, and different shifted pieces may interleave differently after each update. Consequently, a standard lazy segment tree can maintain sums, minima, and maxima, but it cannot maintain exact pairwise absolute differences without additional information about the value distribution.

This obstruction is the starting point of the paper. We treat the dynamic pairwise Manhattan problem as an interaction-routing problem rather than as a single segment-tree aggregate. Each query interval is decomposed into interactions between pieces. Some interactions are easy because their value ranges are separated and the absolute value has a fixed sign. Some are reusable because the same pair of structural objects appears repeatedly. Some require exact range-searching machinery. Others can be evaluated approximately by replacing the absolute-value kernel with a finite-dimensional translation-closed feature map.

The resulting framework is called the *Magnus Model*. The name refers to the structural view used throughout

the paper: the data structure maintains not only interval summaries, but also a routing layer that decides which evaluator is appropriate for each interaction. In low-fragmentation or value-separated regimes this can avoid the naive enumeration of all shifted subinterval pairs; in highly adversarial regimes it falls back to sorting, rebuilding, or pair-space exact evaluation.

► **Theorem 1 (Informal exact word-RAM baseline).** *For fixed-width integer values, a lazy range-add tree can extract the m current values of a query interval in $O(m + \log n)$ time. Radix-statistical order recovery followed by the sorted-rank formula computes the exact pairwise L_1 sum in linear additional time. Thus updates take $O(\log n)$, queries take $O(m + \log n)$, and persistent space is $O(n)$, with $O(m)$ query workspace.*

► **Theorem 2 (Informal pair-space compilation benchmark).** *For the one-dimensional exact problem, a static pair-space orthogonal range-searching oracle combined with epoch rebuilding supports operations in*

$$O(n^{4/3} \text{polylog } n)$$

amortized time after $O(n^2 \text{polylog } n)$ pair-space compilation. This is a preprocessing–query tradeoff, not an improvement over linear order recovery for ordinary large instances.

► **Theorem 3 (Informal exact hierarchical factorization).** *Fix g shifted runs and let C_{st} be the total absolute-distance interaction between runs s and t . There is an exact factorization*

$$C = \sum_{\nu} B_{\nu}, \quad \text{rank}(B_{\nu}) \leq 4,$$

indexed by binary value-domain splits. If η_Q is the total number of active run appearances over these splits and range count/sum statistics cost $R(n)$, the factors can be constructed in $O(\eta_Q R(n))$ time and stored in $O(\eta_Q)$ space. For a fixed factorization with h_Q nonzero factor nodes, B subset queries over the same runs can be evaluated in

$$O(\min\{B\eta_Q, \text{MM}(B, g, h_Q) + Bh_Q\})$$

time.

► **Theorem 4 (Informal lazy-incremental transform).** *For integer multisets, let $P_A(z) = \sum_x h_A(x)z^x$ and $Q_{A,B}(z) = P_A(z^{-1})P_B(z)$. A full translation acts by a monomial, while translating a submultiset S by Δ changes the histogram by $(z^{\Delta} - 1)P_S(z)$. The discrete second difference of $H_{A,B}(t) = \sum_{a,b} |b - a + t|$ is twice the corresponding coefficient of $Q_{A,B}$. Therefore full-cover updates are purely lazy, whereas partial updates induce exact bilinear correlation deltas that can be buffered and periodically compacted by the hierarchical factorization.*

► **Theorem 5 (Informal concrete shift–meld structure).** *Let d_u be the number of distinct values in a segment-tree node and let*

$$\mathcal{J}(a, b) = \min\{a, b\} \log \left(\frac{\max\{a, b\}}{\min\{a, b\}} + 1 \right)$$

be the work of a join-based histogram union. A persistent weighted-map segment tree supports a range addition on I in

$$O\left(\log n + \sum_{u \in \partial I} \mathcal{J}(d_{u_L}, d_{u_R})\right)$$

work, where ∂I is the set of partially covered nodes; this is $O(n)$ in the worst case. If a query of length M decomposes into $k = O(\log n)$ canonical nodes with total distinct support D_Q , it is answered exactly in

$$O(\log n + \min\{M, k + D_Q \lceil \log_2 k \rceil\})$$

time for fixed-width integers, using $O(n \log n)$ live space.

► **Theorem 6 (Informal adaptive-route refinements).** Let a query contain a clean multiset C , dirty proxy values O_D , and true dirty values V_D . The exact correction satisfies

$$\begin{aligned} F(C \cup V_D) - F(C \cup O_D) &= F(V_D) - F(O_D) \\ &\quad + \text{Cross}(C, V_D) \\ &\quad - \text{Cross}(C, O_D), \end{aligned}$$

so fixed-width dirty correction costs $O(D_Q R'(n) + D_Q)$, rather than $O(D_Q R'(n) + \text{Ord}(D_Q))$. A compiled interaction with p sorted breakpoints supports a sequence of relative-offset evaluations in

$$O\left(q + \sum_{t=2}^q \log(1 + |k_t - k_{t-1}|)\right)$$

time by kinetic finger search, and in $O(q + p)$ time for monotone offsets. If cold evaluation costs C_e , compiled evaluation costs $q_e < C_e$, and compilation costs B_e , a rent-or-buy threshold gives a two-competitive online policy relative to the better of always-cold and compile-immediately strategies.

► **Theorem 7 (Informal conditional barrier).** Let an exact one-dimensional data structure on positive integer arrays have preprocessing time $P(n)$, update time $U(n)$, and query time $Q(n)$. Then Tripartite 3SUM on three integer sets of size n can be solved in

$$P(2n) + O(n(U(2n) + Q(2n)) + n \log n)$$

time. Consequently, under the Tripartite 3SUM Hypothesis, truly subquadratic preprocessing and truly sublinear update-plus-query time cannot hold simultaneously. The reduction remains valid when every update targets one fixed half of the array and every query targets the whole array.

► **Theorem 8 (Informal spectral benchmark).** Suppose a D -term translation-closed kernel approximates $|u|$ with uniform error at most ε on the relevant one-dimensional difference range. Then a Lazy Spectral Segment Tree supports range additions and approximate range pairwise L_1 queries in

$$O(D \log n)$$

time and $O(Dn)$ space per coordinate, with additive error at most $\varepsilon \binom{m}{2}$ on a one-dimensional query of length m .

The eight informal statements delimit the exact and approximate landscape. Fixed-width radix aggregation removes the comparison-sorting logarithm but still performs essentially linear order recovery. Pair-space compilation shows how near-quadratic preprocessing can buy much faster static Cross evaluation. The hierarchical factorization identifies the exact matrix structure already computed by value-domain divide and conquer and exposes a batch-matrix-multiplication route when many selectors share one shifted-run geometry. The lazy-incremental transform explains how translations act before a value decomposition is chosen, while the shift-meld structure gives a complete support-sensitive realization with explicit update, query, and space bounds. The adaptive refinements remove an avoidable quadratic dirty term and turn repeated Cross evaluation into a locality-sensitive compiled route. The conditional barrier shows that the parametric cross interaction already encodes Tripartite 3SUM, even under severely restricted updates and queries, and therefore explains why the remaining linear dependence is difficult to remove. The spectral benchmark shows what becomes possible once exactness is relaxed and the absolute-value cusp is replaced by translation-stable finite-dimensional algebra.

A motivating interpretation is dynamic regional dispersion. Points may represent spatial or state features of map cells, sensors, or ordered events, and a query value measures the total internal L_1 dispersion of a selected region. A range translation models a coherent perturbation applied to a contiguous region, such as a common shift in load, pressure, calibration, risk score, or normalized state. This interpretation is not needed for the algorithms, but it explains why pairwise dispersion under coherent interval updates is a natural aggregate to maintain.

1.1 Contributions

The paper makes the following contributions.

1. **Reduction to the one-dimensional core.** We isolate the one-dimensional dynamic pairwise L_1 aggregation problem as the essential component of the three-dimensional Manhattan problem. This separates the geometric statement of the problem from the order-maintenance difficulty that drives the data structure.
2. **Linear exact order recovery for word integers.** We combine lazy range-add extraction with radix-statistical sorting and give a direct radix divide-and-aggregate formulation that never emits the fully sorted sequence. A query of length m is answered exactly in $O(m + \log n)$ time for fixed-width integers, while updates take $O(\log n)$. This removes the comparison-sorting logarithm without pretending to avoid processing essentially all values in adversarial instances.
3. **Pair-space compilation tradeoff.** We lift each ordered pair (i, j) to $(i, j, a_j - a_i)$, reducing exact Cross queries to weighted three-dimensional orthogonal range searching. Rebuilding the compiled oracle after bounded epochs gives $O(n^{4/3} \text{polylog } n)$ amor-

tized operations after $O(n^2 \text{polylog } n)$ preprocessing. Its role is to expose the price of precompiling pair-difference breakpoints, not to supersede the linear radix baseline.

4. **Segment-tree amortized exact spine.** We clarify that the practical exact route is not the quadratic pair-space oracle itself, but a segment-tree-centered interaction structure: full-cover translations preserve node-internal pairwise values, while only cross-node interactions and dirty residues require heavy evaluation.
5. **Exact hierarchical, lazy-incremental, and shift-meld kernel structures.** For shifted runs with frequency matrix H , we identify the exact interaction matrix $C = HKH^T$, where $K_{pq} = |v_p - v_q|$. Ordered separated blocks of K have rank at most two, and a binary value tree decomposes C into rank-at-most-four local factors. We further show that translations act monomially on Laurent histograms and by shifts on pair-difference correlations, while a partial translation creates the exact two-term delta $(z^\Delta - 1)P_S(z)$. Finally, we instantiate these identities by a persistent weighted-map segment tree whose full-cover updates are lazy shifts and whose partial pulls are work-optimal histogram melds, giving explicit distinct-support-sensitive operation bounds and a linear worst-case ceiling.
6. **Fragmentation-aware exact routing and adaptive reuse.** We introduce a refined exact model whose cost parameters are offset fragmentation g , value fragmentation η_Q , interaction-factor sparsity ζ_Q , hierarchical index expansion χ_Q , hard-overlap residue ω_Q , dirty mass D_Q , and raw query length M . Dirty regrouping removes the former D_Q^2 correction term; index-boundary entropy controls uniform-frontier extraction; compiled Cross functions use kinetic finger search; and a rent-or-buy rule decides when repeated interactions justify breakpoint compilation.
7. **A direct 3SUM-based conditional barrier.** We reduce Tripartite 3SUM to a sequence of exact range-translation and pairwise- L_1 queries. The discrete second difference of the cross function recovers the number of pairs with a prescribed sum. Thus, under the Tripartite 3SUM Hypothesis, a structure cannot simultaneously have truly subquadratic preprocessing and truly sublinear update-plus-query time. The hardness persists for positive integers, one fixed updated half, and whole-array queries.
8. **Algebraic CDQ approximation.** We formulate a general translation-closed feature framework in which $|x - y|$ is approximated by $\Phi(x)^T A \Phi(y)$. Exact CDQ cross terms are then replaced by bilinear interactions between feature sums.
9. **Lazy spectral specialization.** Fourier features yield a concrete Lazy Spectral Segment Tree. Range addition becomes multiplication by a complex phase, and a range query is recovered from the squared modulus of frequency sums. This gives an $O(D \log n)$ -time approximate path under an additive ker-

nel approximation guarantee. A refined exact-far/approximate-near decomposition spends no error on separated rank-two blocks and bounds the remaining error by the number and width of the near-cusp pairs actually approximated.

10. **Integrated interaction routing.** We combine exact evaluators, algebraic approximations, sieve-like pruning, dirty correction, and memoization into one routing model. The model uses exact rank-two closed forms whenever signs are determined, kinetic or compiled Cross objects for reused hard interactions, low-rank approximation only on unresolved near-cusp residues, and heavy exact machinery when no structural compression applies.

1.2 Technical Overview

The main exact obstacle is the following cross interaction. Suppose two index pieces I and J have acquired different lazy offsets. For each base pair $(i, j) \in I \times J$, the contribution has the form

$$|a_j - a_i + \lambda|,$$

where λ is the offset difference between the two pieces. If all values $a_j - a_i + \lambda$ have the same sign, the absolute value disappears and the interaction is a closed-form count/sum expression. If the sign changes inside the interaction, the query asks for a threshold count and threshold sum over the pair values $a_j - a_i$. This is exactly the point at which pair-space range searching appears.

The simplest exact route does not attempt to maintain the dynamic order persistently. It extracts the current values of the query interval, recovers their order, and evaluates the sorted-rank formula. In the comparison model this costs $O(m \log m)$; for fixed-width integers, radix-statistical sorting reduces the order-recovery step to $O(m)$. This linear route becomes the unconditional practical fallback and, together with the lower bound below, the reference point for the exact frontier.

The pair-space oracle materializes the same ordering obstruction in compiled form. For a fixed base array, every ordered pair (i, j) , $i < j$, is represented as a three-dimensional point

$$(i, j, a_j - a_i).$$

A Cross query becomes a small number of orthogonal range queries returning counts and weighted sums below a threshold. Static queries are therefore polylogarithmic after $O(n^2 \text{polylog } n)$ preprocessing. In the dynamic setting, we keep a base array for one epoch and represent pending range additions as a small set of offset-constant pieces. A query then expands into Cross queries between these pieces. Balancing the epoch length gives the $O(n^{4/3} \text{polylog } n)$ pair-space epoch benchmark.

The raw epoch method is too coarse because it pays for all pairs of offset pieces. The practical exact architecture therefore keeps the segment tree as the amortized spine. A full-cover translation of a canonical node preserves its internal pairwise answer, so only boundary dirtiness and cross-node interactions remain hard. The fragmentation-aware model refines this spine by asking

which interactions are actually hard. If the number of offset runs is small, run-pair Cross queries are sufficient. If many offset runs collapse into few value fragments, shifted-run value-CDQ is better. If the index decomposition is hierarchical, index-CDQ can reduce repeated work. If value intervals are separated, sieve pruning routes the interaction to a closed form. If only a small set of elements was touched by partial updates, dirty correction isolates their contribution and reuses the clean structure. Regrouping the dirty proxy and true multisets expresses the correction through two small internal sums and two dirty-clean Cross terms, eliminating the previous quadratic dirty-dirty repair. Repeated clean interactions are compiled only after a rent-or-buy threshold and then evaluated kinetically as the relative lazy offset moves through their sorted breakpoints.

The value-domain recursion has an exact matrix interpretation. If H stores the value frequencies of the shifted runs and $K_{pq} = |v_p - v_q|$, then all run-to-run interactions form the matrix $C = HKH^\top$. Whenever two ordered value blocks are separated, the corresponding block of K is the difference of two rank-one matrices and therefore has rank at most two. A binary value tree consequently writes C as a sum of local rank-at-most-four matrices. Scalar value-CDQ is the contraction of these factors against the all-ones vector; retaining the sparse factors permits repeated selectors to be processed together by rectangular matrix multiplication. Thus exact CDQ, separation pruning, and low-rank algebra are not independent mechanisms: they are three views of the same ordered absolute-distance kernel.

The matrix factorization admits a translation-equivariant refinement. Encode an integer value multiset by the Laurent histogram $P_A(z) = \sum_x h_A(x) z^x$. A common translation multiplies P_A by z^Δ , while translating only a submultiset S adds the two-term delta $(z^\Delta - 1)P_S(z)$. The product $P_A(z^{-1})P_B(z)$ is the pair-difference correlation, and its coefficients are exactly the discrete curvature of the translated cross function. Equivalently, $|x - y|$ is a squared distance between exact threshold features; partial translations become box convolutions and produce a compactly supported tent-kernel quadratic correction. These identities make full-cover motion purely lazy and reduce partial updates to structured bilinear deltas that can be buffered and compacted by Matrix-CDQ.

A direct materialized implementation stores each segment-tree histogram in a persistent join-based ordered map augmented by element count, weighted sum, distinct support, and internal pairwise distance. A common translation is a root key-shift tag. A partial ancestor is pulled by work-optimal union of the two child maps, and a range query temporarily melds the $O(\log n)$ canonical maps. This shift-meld route closes the end-to-end exact data-structure loop: it is support-sensitive on repeated-value states, has $O(n)$ worst-case update work and a linear radix query fallback, and provides a concrete flush primitive for the symbolic component buffer.

The same cross function also yields a conditional barrier. For two blocks encoding sets A and B , a translation

parameter t produces

$$H(t) = \sum_{a \in A, b \in B} |a + b - t|.$$

On integer inputs, the discrete second difference $H(t+1) - 2H(t) + H(t-1)$ is exactly twice the number of pairs satisfying $a + b = t$. Querying this quantity at targets $t = -c$ reduces Tripartite 3SUM to the dynamic problem. This shows that the ordering barrier is not only representational: under the Tripartite 3SUM Hypothesis, a broadly efficient exact structure would imply a truly subquadratic 3SUM algorithm.

The approximate side changes the representation rather than the decomposition. The ordered kernel already has exact rank two on every separated value block, so approximation is reserved for unresolved near-cusp blocks rather than spent on the far field. This localizes the additive error to the number of pairs actually sent to spectral evaluation. Instead of maintaining exact breakpoints of $|x - y|$, choose a feature map Φ for which translations are linear:

$$\Phi(x + \Delta) = T_\Delta \Phi(x).$$

If $|x - y| \approx \Phi(x)^\top A \Phi(y)$, then the cross contribution between two pieces is approximately

$$S_L^\top A S_R, \quad S_L = \sum_{i \in L} \Phi(a_i), \quad S_R = \sum_{j \in R} \Phi(a_j).$$

Thus the same CDQ split used by the exact model can be evaluated algebraically. Fourier features are especially convenient because a translation is only a rotation or complex phase multiplication.

A useful way to read the paper is therefore as a complexity map. Linear radix aggregation is the direct exact endpoint for fixed-width integers. Pair-space range searching trades near-quadratic compilation for fast Cross evaluation. Fragmentation parameters explain when exact interactions are easier than the adversarial linear fallback. The 3SUM reduction places a conditional wall behind strongly sublinear exact queries with light preprocessing. Algebraic CDQ and spectral features cross that wall only by accepting approximation. Sieve pruning and memoization remain acceleration layers for structured workloads rather than substitutes for the exact worst-case frontier.

1.3 Organization

Section 2 discusses related work and positions the model among integer sorting, dynamic range queries, conditional lower bounds, range searching, hierarchical matrices, divide-and-conquer methods, and feature approximations. Section 3 formalizes the problem and the coordinatewise reduction. The subsequent exact sections develop comparison and radix order recovery, pair-space compilation, the segment-tree interaction spine, fragmentation-aware routing, kinetic rent-or-buy compilation, dirty regrouping, shifted-run value-domain divide and conquer, its exact hierarchical matrix factorization, the lazy-incremental

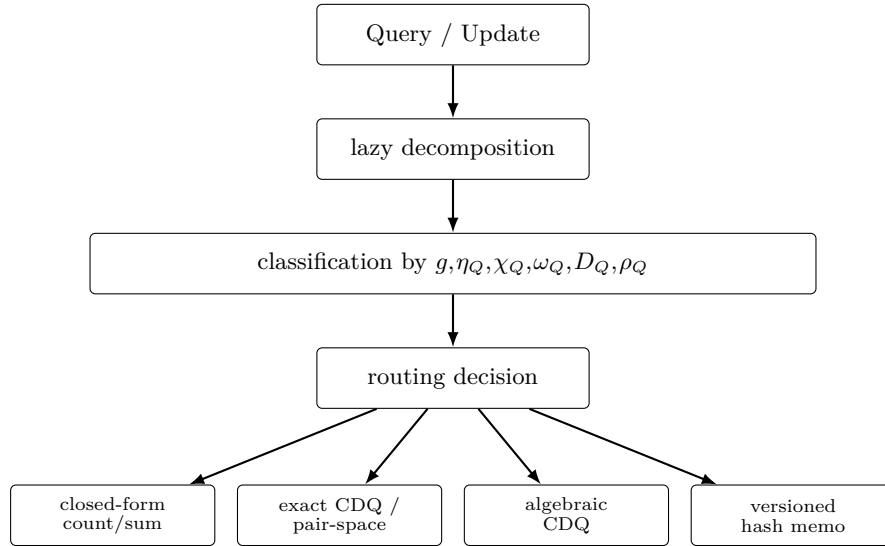


Figure 1: Closed interaction-routing view of the Magnus Model. Interactions are classified by fragmentation, dirty perturbation, and value geometry, then routed to closed-form, exact, approximate, or memoized evaluators.

curvature and histogram transforms, a concrete shift-meld histogram segment tree, index-domain decomposition, and rebuilding policies. The ordering-barrier section then proves the Tripartite-3SUM conditional tradeoff. The approximate sections develop translation-closed algebraic features and the spectral segment tree, followed by kernel choice, exact-far/approximate-near routing, and integrated routing. The final sections synthesize the frontier, implementation consequences, limitations, applications, and open directions.

2 Related Work and Positioning

The problem sits at the intersection of dynamic range data structures, geometric range searching, divide-and-conquer evaluation, and kernel approximation. The point of this section is not to give an exhaustive survey, but to clarify which parts of the paper use standard machinery and which parts are specific to dynamic pairwise L_1 aggregation.

Dynamic range aggregates. Classical range-sum structures such as Fenwick trees and segment trees are highly effective when the maintained aggregate is composable [8]. Lazy propagation extends this paradigm to many interval updates, but the paradigm fundamentally relies on summaries that can be merged from children. The pairwise L_1 objective is different: when two subintervals are merged, the cross contribution between them is itself a nontrivial query. The Magnus Model can be seen as making this missing cross term explicit.

Static value-order structures. Merge-sort trees, wavelet-tree-like indexes, and related range-searching structures can maintain sorted information for static or partially dynamic arrays. They are natural baselines for pairwise absolute differences because, once the values in a query interval are sorted, the answer is a prefix-sum

expression. The difficulty considered here is that range translations shift multiple subranges by different offsets, causing their value orders to interleave dynamically.

Integer sorting and radix aggregation. For bounded machine integers, comparison sorting is not the correct baseline. Classical radix sorting recovers the exact order in a constant number of stable counting passes when the word width is fixed [16]. Section 4.2 combines this fact with lazy range extraction. The resulting method is deliberately elementary: it establishes a linear exact upper bound and clarifies that the remaining obstacle is not the comparison-sorting logarithm, but the need to process essentially all query values in adversarial instances.

Join-based ordered maps. Work-optimal union, intersection, and difference for balanced ordered sets can be implemented from a generic join primitive in $O(m \log(n/m+1))$ work for input sizes $m \leq n$ [7]. The shift-meld structure applies this machinery to weighted multisets rather than plain sets: equal keys combine multiplicities, subtree summaries maintain the exact pairwise-distance aggregate, and path copying plus a lazy key-shift tag preserves child maps while rebuilding partially changed ancestors.

Geometric range searching. The pair-space exact oracle uses a standard geometric lifting. Each ordered pair becomes a point in a three-dimensional space, and a thresholded absolute-value query becomes a weighted orthogonal range query. Orthogonal range searching and its relatives are classical topics in computational geometry [4, 13]. The contribution here is not a new static range-searching primitive, but the observation that dynamic pairwise L_1 aggregation can be reduced to such primitives inside an epoch framework.

3SUM-based conditional lower bounds. The 3SUM problem and its tripartite variants are standard sources of fine-grained conditional lower bounds [17, 18, 19]. Pătraşcu initiated the use of 3SUM to obtain polynomial conditional lower bounds for dynamic problems, and subsequent work strengthened this framework. The reduction in Section 13 is direct and problem-specific: it uses the discrete curvature of a translated absolute-value cross function to recover exact pair-sum multiplicities. No hashing or intermediate set-disjointness instance is required.

Divide and conquer over interactions. The CDQ-style parts of the model follow the broad paradigm of multidimensional divide and conquer [9]. In this paper the phrase “CDQ” is used descriptively: it denotes a recursive decomposition in which internal contributions are solved in subproblems and cross contributions are evaluated separately. Exact CDQ evaluates the cross terms by range searching, value-domain decomposition, or pruning. Algebraic CDQ evaluates the same cross terms by bilinear forms between feature sums.

Structured matrix products as methodological precedent. Deng, Kirkpatrick, Rong, Vassilevska Williams, and Zhong obtained a faster additive approximation for unweighted all-pairs shortest paths by exposing a hidden min-plus product and ordering vertices along an Euler tour so that the induced distance matrices have bounded differences [11]. Their graph problem, approximation guarantee, and technical reduction are different from those studied here, and no theorem in this manuscript depends on their algorithm. Nevertheless, their methodological pattern—rewrite a costly all-pairs interaction as an algebraic product, then search for exploitable structure in the induced matrices—motivated our investigation of hidden structure in dynamic Cross interactions and of the boundary between exact order recovery and algebraic approximation.

Hierarchical low-rank matrices. Hierarchical matrix methods represent large matrices through low-rank descriptions of suitably separated blocks [12]. The exact factorization developed here is specialized to the one-dimensional absolute-distance kernel and does not invoke numerical compression: after ordering the value atoms, every strictly separated kernel block has algebraic rank at most two. The value-domain recursion lifts these rank-two kernel blocks to sparse rank-at-most-four factors over shifted runs.

Kernel and feature approximations. The approximate track is related to feature representations for kernels, including random-feature methods [3] and deterministic approximation theory [5, 6]. The data-structural requirement is narrower than general kernel learning: we need a finite expansion that approximates $|x - y|$ on the relevant scale and remains stable under translations. Fourier, polynomial, Chebyshev, multiscale, and hybrid features

are different ways of satisfying this requirement with different error and stability tradeoffs.

Filtering and pruning. The sieve-like pruning component is conceptually related to filtering search in geometric data structures [15]. Before invoking a heavy evaluator, the model asks whether the local value geometry already decides the interaction. If two shifted value intervals are separated, the absolute value has a fixed sign and the exact cross contribution collapses to a count/sum formula. If an interaction is far from the cusp of $|x - y|$, the ordered separated block is already exact rank two and is evaluated by counts and sums. Spectral approximation is reserved for unresolved near-cusp blocks whose exact factorization is still expensive. Only that near-overlap residue pays approximation error or heavier exact machinery.

3 Problem Formulation

► **Problem 1 (Dynamic range pairwise Manhattan sum).** Maintain a sequence of points

$$P_1, \dots, P_n \in \mathbb{R}^3, \quad P_i = (x_i, y_i, z_i).$$

Support:

- Add($L, R, \Delta_x, \Delta_y, \Delta_z$): for all $i \in [L, R]$,

$$x_i \leftarrow x_i + \Delta_x, \quad y_i \leftarrow y_i + \Delta_y, \quad z_i \leftarrow z_i + \Delta_z.$$

- Query(l, r):

$$\text{Ans}(l, r) = \sum_{l \leq i < j \leq r} (|x_i - x_j| + |y_i - y_j| + |z_i - z_j|).$$

The reduction from the three-dimensional statement to the one-dimensional core is elementary, but it is useful to make the two roles of the reduction explicit: updates decouple coordinatewise, and queries decouple by additivity of the Manhattan metric.

► **Lemma 1 (Update decoupling).** *A three-dimensional range translation is exactly three independent one-dimensional range additions. More precisely, the operation*

$$P_i \leftarrow P_i + (\Delta_x, \Delta_y, \Delta_z) \quad (i \in [L, R])$$

is equivalent to applying $x_i \leftarrow x_i + \Delta_x$, $y_i \leftarrow y_i + \Delta_y$, and $z_i \leftarrow z_i + \Delta_z$ on the same index interval.

Proof. The three coordinates of P_i are updated independently and no coordinate participates in the update rule of another coordinate. Therefore any data structure that supports the one-dimensional update can be run once for each coordinate. $\square$

► **Lemma 2 (Coordinate decomposition).** *For any interval $[l, r]$,*

$$\text{Ans}(l, r) = F_x(l, r) + F_y(l, r) + F_z(l, r),$$

where

$$F_x(l, r) = \sum_{l \leq i < j \leq r} |x_i - x_j|,$$

and F_y, F_z are defined analogously.

Proof. For every pair (i, j) , the Manhattan distance is

$$\|P_i - P_j\|_1 = |x_i - x_j| + |y_i - y_j| + |z_i - z_j|.$$

Summing this identity over all $l \leq i < j \leq r$ and regrouping the three sums gives the claimed equality. $\square$

► **Corollary 1 (One-dimensional sufficiency).** *If the one-dimensional dynamic pairwise L_1 problem can be solved with update cost $U(n)$, query cost $Q(n)$, and space $S(n)$, then the three-dimensional Manhattan problem can be solved with update and query costs multiplied by a factor of three and with space $3S(n)$.*

Proof. Maintain one independent copy of the one-dimensional structure for each coordinate. The update-decoupling statement handles updates, and the coordinate-decomposition statement handles queries. $\square$

Therefore, it suffices to study the one-dimensional dynamic problem.

► **Definition 1 (One-dimensional dynamic pairwise L_1 problem).** Maintain a sequence $a_1, \dots, a_n \in \mathbb{R}$ under:

■ Add(L, R, Δ): for all $i \in [L, R]$,

$$a_i \leftarrow a_i + \Delta.$$

■ Query(l, r):

$$F(l, r) = \sum_{l \leq i < j \leq r} |a_i - a_j|.$$

4 Exact Algorithms

We first discuss exact algorithms. The goal of this section is not only to give baselines, but also to isolate the structural parameters that make exact dynamic maintenance difficult.

Throughout this section we work with the one-dimensional problem

$$F(l, r) = \sum_{l \leq i < j \leq r} |a_i - a_j|.$$

The three-dimensional Manhattan answer is obtained by applying the same method to the three coordinates independently and summing the results.

4.1 Sorting-Based Exact Query

For a query interval $[l, r]$, let

$$m = r - l + 1.$$

The most direct exact method is to materialize the current values in the interval, sort them, and then use the rank of each value in the sorted order. The following two lemmas separate the rank-counting fact from the final formula.

► **Lemma 3 (Contribution of one sorted value).** *Let $v_1 \leq v_2 \leq \dots \leq v_m$. In the expansion of*

$$\sum_{1 \leq i < j \leq m} (v_j - v_i),$$

the coefficient of v_k is $2k - m - 1$.

Proof. The value v_k appears positively in all pairs (i, k) with $i < k$, so it contributes $k - 1$ positive copies. It appears negatively in all pairs (k, j) with $k < j$, so it contributes $m - k$ negative copies. The net coefficient is therefore

$$(k - 1) - (m - k) = 2k - m - 1.$$

$\square$

► **Lemma 4 (Sorted-value formula).** *Let $v_1 \leq v_2 \leq \dots \leq v_m$. Then*

$$\sum_{1 \leq i < j \leq m} |v_i - v_j| = \sum_{k=1}^m (2k - m - 1)v_k.$$

Proof. Since the values are sorted, $|v_i - v_j| = v_j - v_i$ for all $i < j$. Applying the previous rank-contribution lemma to the resulting signed expansion gives the formula. $\square$

This gives a direct exact query algorithm: extract all values in $[l, r]$, sort them, and apply the formula. It is deliberately simple, but it will reappear later as the adversarial fallback of the adaptive planner.

► **Theorem 9 (Direct exact baseline).** *If range additions are represented by ordinary lazy propagation and a query extracts all current values in $[l, r]$, then an update takes $O(\log n)$ time and a comparison-based query of length m takes*

$$O(m \log m + \log n)$$

time.

This baseline is important because it is the final fallback of the adaptive exact model for arbitrary comparable values. For bounded machine integers, the comparison logarithm can be removed.

4.2 Radix-Statistical Exact Query for Word Integers

Assume that every current value is represented by a signed w -bit machine word and that updates do not overflow this representation. Signed order is converted to unsigned lexicographic order by flipping the sign bit. A stable least-significant-digit radix sort with b -bit digits then uses

$$d = \left\lceil \frac{w}{b} \right\rceil$$

counting passes. Each pass costs $O(m + 2^b)$ time and uses $O(m + 2^b)$ workspace. After the values are sorted, the sorted-value formula computes the answer in one scan.

The term “radix-statistical” emphasizes that each pass uses exact bucket counts rather than comparisons or lossy sketches. Equivalently, one may view the computation as recursively separating digit buckets and using their counts and sums to account for all cross-bucket pairs. Either view recovers the same exact order information.

► **Lemma 5 (Lazy extraction).** *A standard lazy segment tree storing the base value at each leaf and an additive tag at each internal node can output all current values in an interval of length m in $O(m + \log n)$ time.*

Proof. Traverse only nodes intersecting the query interval and carry the accumulated lazy tag down the recursion. The traversal contains $O(\log n)$ boundary nodes

and $O(m)$ nodes in the minimal subtree spanning the reported leaves. Each reported leaf is visited once. $\square$

► **Theorem 10 (Radix-statistical exact baseline).** *For signed w -bit integer values, the one-dimensional dynamic problem admits*

$$U(n) = O(\log n)$$

update time and, for a query interval of length m ,

$$Q(n, m) = O\left(m + \log n + \left\lceil \frac{w}{b} \right\rceil (m + 2^b)\right)$$

query time, using $O(n)$ persistent space and $O(m + 2^b)$ query workspace. For fixed word width and fixed digit width, this is $O(m + \log n)$ time. For a whole-array query in the standard $w = \Theta(\log n)$ word-RAM, choosing $b = \Theta(\log n)$ with $2^b = O(n)$ also gives $O(n)$ time.

Proof. The update bound is ordinary lazy propagation. By the lazy-extraction lemma, the query values are materialized in $O(m + \log n)$ time. Radix sorting takes the stated number of counting passes, after which the sorted-value formula takes $O(m)$ time. The persistent tree is linear; the radix buffers and counters are temporary. $\square$

This theorem does not show that order can be avoided. It shows that, for bounded integers, exact order recovery can be implemented in linear rather than comparison-sorting time. Section 13 will show that this linear full-range dependence is conditionally tight under light preprocessing and fast updates.

4.2.1 Direct radix divide-and-aggregate

The sorted sequence is not itself required: only the sum of all cross-bucket differences is needed. After flipping the sign bit, consider a binary most-significant-bit recursion. At a recursion node all values share the already processed prefix. Split the multiset into S_0 and S_1 according to the next order-key bit, and let (n_0, s_0) and (n_1, s_1) be their counts and sums of the original signed values. The sign-flipped words are used only as order keys. Every value in S_0 precedes every value in S_1 , so their entire cross contribution is

$$n_0 s_1 - n_1 s_0.$$

The algorithm adds this term and recurses independently inside S_0 and S_1 .

► **Lemma 6 (Unique radix separation).** *Every unequal pair of machine words is charged exactly once by the direct radix recursion, at its most significant differing bit; equal pairs are never charged and contribute zero.*

Proof. Before the most significant differing bit, the two words share a recursion node. At that bit one enters the zero child and the other the one child, so the cross-bucket formula charges their larger-minus-smaller difference. They never share a descendant. Equal words remain together through all bits and have zero distance. $\square$

► **Theorem 11 (Direct radix aggregation).** *Given m signed w -bit values, their exact total pairwise L_1 distance*

can be computed without producing a sorted output in $O(wm)$ time and $O(m + w)$ workspace. Hence the cost is $O(m)$ for fixed-width words. A fixed b -bit generalization replaces the binary split by ordered radix buckets and uses $O(\lceil w/b \rceil m)$ time.

Proof. At each bit depth, every active value participates in one stable partition and one count/sum accumulation. The total work per depth is $O(m)$, and there are at most w depths. The correctness lemma partitions all unequal pairs by their first separating digit. A reusable partition buffer and the recursion stack give the workspace bound. $\square$

This formulation has the same fixed-word asymptotic bound as radix sorting, but it exposes the same structural principle used later by value-CDQ: once two ordered buckets are separated, their complete interaction is a count/sum expression and no internal ordering across the buckets is required.

4.3 Offset-Merge Epoch Baseline

At the beginning of an epoch, let the current array be fixed as

$$base_1, \dots, base_n.$$

Build a merge-sort tree on the base array. During the epoch, range additions are stored as pending updates

$$(L_s, R_s, \Delta_s).$$

When the number of pending updates reaches a threshold T , all pending updates are applied to the base array and the merge-sort tree is rebuilt.

A query interval $[l, r]$ is split by the endpoints of the pending updates into at most $2T + 1$ offset-constant subintervals

$$I_1, I_2, \dots, I_k.$$

For every $i \in I_p$,

$$a_i = base_i + o_p$$

for some offset o_p . Each I_p is decomposed into canonical nodes of the merge-sort tree. Hence the values in $[l, r]$ are represented as shifted sorted lists. A multiway merge gives the sorted current values, and the sorted-value formula gives the exact answer.

► **Theorem 12 (Exact offset-merge tree).** *Let T be the epoch length. The offset-merge tree supports updates with amortized time*

$$O\left(\frac{n \log n}{T}\right)$$

and answers a query of length m in

$$O(T \log n + m \log(T \log n))$$

time.

Proof. Rebuilding costs $O(n \log n)$, amortized over T updates. The query is split into $O(T)$ offset-constant intervals, each decomposed into $O(\log n)$ canonical merge-sort-tree nodes. The multiway merge of m values from $O(T \log n)$ shifted sorted lists costs $O(m \log(T \log n))$. $\square$

4.4 Pair-Space Exact Oracle

Fix a base array $a_1, \dots, a_n$. For each pair $i < j$, define

$$d_{ij} = a_j - a_i.$$

For two index intervals I, J and a shift λ , define

$$\text{Cross}(I, J, \lambda) = \sum_{\substack{i \in I, j \in J \\ i < j}} |d_{ij} + \lambda|.$$

Let R be the corresponding rectangle in the (i, j) -plane, restricted to valid pairs $i < j$. The only nonlinearity in Cross is the sign change of $d_{ij} + \lambda$, so the query is controlled by a single threshold in the third coordinate.

► **Definition 2 (Threshold statistics for Cross).** For $\theta = -\lambda$, define

$$N = \#\{(i, j) \in R : i < j\}, \quad S = \sum_{(i, j) \in R, i < j} d_{ij},$$

$$K = \#\{(i, j) \in R : i < j, d_{ij} \leq \theta\},$$

$$U = \sum_{(i, j) \in R, i < j, d_{ij} \leq \theta} d_{ij}.$$

Here N, S describe all valid pairs in the index rectangle, while K, U describe only the pairs lying below the sign-change threshold.

► **Lemma 7 (Absolute-value threshold split).** For any finite multiset D of real numbers and any shift λ , let

$$K = \#\{d \in D : d \leq -\lambda\}, \quad U = \sum_{d \in D, d \leq -\lambda} d,$$

$$N = |D|, \quad S = \sum_{d \in D} d.$$

Then

$$\sum_{d \in D} |d + \lambda| = S - 2U + (N - 2K)\lambda.$$

Proof. Elements with $d \leq -\lambda$ contribute $-(d + \lambda)$, and the remaining elements contribute $d + \lambda$. Therefore

$$\begin{aligned} \sum_{d \in D} |d + \lambda| &= -(U + K\lambda) + (S - U) + (N - K)\lambda \\ &= S - 2U + (N - 2K)\lambda. \end{aligned}$$

□

► **Lemma 8 (Cross formula).** With the threshold statistics above,

$$\text{Cross}(I, J, \lambda) = S - 2U + (N - 2K)\lambda.$$

Proof. Apply the absolute-value threshold split to the multiset $D = \{d_{ij} : (i, j) \in R, i < j\}$. The quantities in the threshold-statistics definition are exactly the four statistics required by the split. □

Therefore exact static Cross queries reduce to weighted three-dimensional orthogonal range searching over pair points

$$(i, j, d_{ij}).$$

There are $O(n^2)$ such points.

► **Theorem 13 (Static pair-space oracle).** For a fixed base array, $\text{Cross}(I, J, \lambda)$ can be answered exactly in

$$O(\text{polylog } n)$$

time using

$$O(n^2 \text{polylog } n)$$

space and preprocessing time.

Proof. Build a weighted static orthogonal range-searching structure on the points (i, j, d_{ij}) , with weight d_{ij} . The structure returns both count and weight sum in an orthogonal box, giving K and U . The values N and S are obtained by the same structure with the d -constraint removed. The Cross formula gives the answer. □

4.5 Epoch Exact Oracle

During an epoch, suppose there are at most T pending updates. A query interval $[l, r]$ is split into at most $2T + 1$ offset-constant intervals

$$I_1, I_2, \dots, I_k,$$

where the offset of I_p is o_p . Then

$$F(l, r) = \sum_{1 \leq p \leq q \leq k} \text{Cross}(I_p, I_q, o_q - o_p).$$

► **Theorem 14 (Epoch exact oracle).** Using the static pair-space oracle, the dynamic one-dimensional exact problem supports updates with amortized time

$$O\left(\frac{n^2 \text{polylog } n}{T}\right)$$

and queries in

$$O(T^2 \text{polylog } n).$$

Proof. Rebuilding the pair-space oracle costs $O(n^2 \text{polylog } n)$, amortized over T updates. A query is split into $O(T)$ offset-constant intervals, hence there are $O(T^2)$ interval pairs. Each pair is answered by one Cross query. □

Taking $T = n^{2/3}$ gives the theoretical bound

$$O(n^{4/3} \text{polylog } n)$$

per operation. This is the basic exact worst-case-style benchmark of the note.

5 Segment-Tree Amortization and the Meaning of the Polynomial Benchmark

The pair-space epoch oracle gives a clean compiled-interaction benchmark, but it is not the intended implementation path for ordinary input sizes and it is not a better worst-case baseline than linear radix order recovery. Its value is conceptual: it shows how near-quadratic preprocessing can precompile the pair-difference events that make exact Cross queries hard. In practical exact data structures, the stronger spine is still a segment tree, not because an ordinary lazy segment tree solves the

problem, but because interval translations are perfectly aligned with segment-tree canonical decomposition. The correct statement is therefore more nuanced: a monoid segment tree is insufficient, while a segment-tree-centered interaction structure is the most natural amortized exact architecture.

A range translation has a special stability property. If a whole object is translated by a common offset, every internal pairwise distance inside that object is unchanged. Thus a full-cover update on a segment-tree node should not rebuild its internal pairwise answer. It should only change a lazy offset attached to the node. The difficulty is pushed to cross interactions between objects with different offsets. This is exactly the role of the Magnus Model: the segment tree supplies the interval skeleton, while the interaction-routing layer decides how to evaluate the non-monoid cross terms.

► **Definition 3 (Segment-tree interaction node).** A clean segment-tree node stores its length, a lazy translation tag, an internal exact pairwise value, min/max shifted value intervals, a version counter, and enough value summary information to support Cross, count/sum, or local rebuilding. Dirty boundary information is stored separately or inside the lowest affected nodes. A full-cover range addition changes the lazy tag and shifted value interval, but does not change the internal pairwise value.

► **Lemma 9 (Full-cover translation stability).** Let $S = (x_1, \dots, x_m)$ be any finite indexed multiset of one-dimensional values and let

$$P(S) = \sum_{1 \leq i < j \leq m} |x_i - x_j|$$

denote its internal pairwise L_1 sum. For every scalar Δ , let $S + \Delta = (x_1 + \Delta, \dots, x_m + \Delta)$. Then

$$P(S + \Delta) = P(S).$$

Proof. Every pair contribution is invariant because

$$|(x + \Delta) - (y + \Delta)| = |x - y|.$$

Summing over all unordered pairs gives the claim. $\square$

This lemma is the reason segment-tree amortization is stronger than a raw epoch decomposition in common regimes. A full-cover update of a canonical node has constant structural effect on that node: the internal answer is preserved, and only the tag changes. Partial updates are localized to $O(\log n)$ boundary paths and may be handled by dirty correction or local rebuild. Queries are also decomposed into $O(\log n)$ canonical nodes. The query answer is the sum of internal answers of these nodes plus the cross interactions between canonical nodes.

► **Theorem 15 (Segment-tree exact reduction).** Consider a one-dimensional exact structure whose query interval $Q = [l, r]$ is decomposed into canonical clean nodes $U_1, \dots, U_k$, with $k = O(\log n)$, plus a dirty residue of size D_Q . Suppose each clean node stores its internal pairwise sum and exposes a Cross evaluator of cost $C(U_p, U_q)$ for

the shifted interaction between two clean nodes. Then the exact query cost can be written as

$$O\left(k + \sum_{1 \leq p < q \leq k} C(U_p, U_q) + D_Q R'(n) + \text{Ord}(D_Q)\right),$$

where $R'(n)$ is the point-to-range correction cost used for dirty-clean interactions.

Proof. The canonical nodes partition the clean part of the query. Internal pairs whose two endpoints lie in the same canonical node are accounted for by that node's stored internal pairwise sum. Every clean pair whose endpoints lie in different canonical nodes belongs to exactly one ordered node pair U_p, U_q and is counted by the corresponding shifted Cross evaluator. Dirty-clean and dirty-dirty pairs are not trusted to the clean summaries and are repaired separately by the stated correction terms. These classes are disjoint and cover all unordered pairs in the query interval, giving the formula. $\square$

The theorem should be read as an architectural reduction, not as a claim that all Cross evaluators are cheap. It says that the ordinary interval work is already minimized: the remaining cost is precisely the cost of evaluating cross interactions between the few structural objects selected by the segment tree. In low-fragmentation inputs, those interactions hit cached functions, separated value intervals, or small value-CDQ subproblems. In adversarial inputs, the structure can still fall back to sorting or rebuilding. Thus the segment tree gives the correct amortization boundary: full-cover motion is free inside a node, while only boundary dirtiness and cross-node ordering remain expensive.

5.1 Why the Segment Tree Is the Practical Exact Spine

Among standard range-update data-structure patterns, the segment tree has four advantages for this problem. First, it matches the update geometry. Every interval update decomposes into $O(\log n)$ full-cover nodes, and full-cover nodes preserve their internal pairwise values. A block decomposition can also exploit full covers, but it has coarser boundaries; an epoch method sees the accumulated update endpoints globally and may fragment unrelated future queries.

Second, the segment tree localizes disorder. A partial update does not globally invalidate the whole representation. It creates dirty mass along boundary paths or inside a small number of rebuilt objects. This is exactly the regime in which dirty correction is useful: a query pays for the clean skeleton plus a small correction, instead of paying for all elements in the range.

Third, the segment tree is compatible with multiple exact evaluators. Each internal node can own a sorted base list, a compressed value summary, min/max bounds, a version number, or a compiled interaction object. Cross interactions between nodes can be routed to closed forms, memoized $H_{A,B}(z)$ functions, value-CDQ, index-CDQ, pair-space range searching, or local sorting. The same

canonical decomposition is reused no matter which evaluator wins.

Fourth, the segment tree has better engineering locality than the quadratic pair-space oracle. It stores summaries on $O(n)$ structural nodes rather than materializing $O(n^2)$ pair points globally. Some node-pair interactions may still be compiled and cached, but they are created only when the workload asks for them. This makes the structure adaptive: repeated hard interactions become cheap, while one-off adversarial interactions do not force a global quadratic preprocessing cost.

For these reasons, the practical exact path is not a single formula such as $O(n^{4/3} \text{polylog} n)$. It is an amortized planner over a segment-tree spine:

range update/query skeleton+local rebuild
+cross-interaction routing.

The skeleton handles what interval data structures are good at. The router handles the part that is not a monoid aggregate.

5.2 Theoretical Role of the Polynomial Exact Oracle

The polynomial pair-space benchmark still matters. It should not be removed merely because it is too large for most implementations. It has three theoretical roles.

First, it is a witness of exact compilation. The linear radix baseline already gives an exact practical upper bound for word integers. Pair-space serves a different purpose: it shows that the breakpoints of exact Cross interactions can be materialized globally so that each compiled interaction is answered polylogarithmically inside an epoch.

Second, it isolates the cost of value ordering. The quadratic preprocessing is not accidental. The absolute-value cross function between two sets has breakpoints at all differences $x - y$. Pair-space materialization pays for those breakpoints explicitly. The segment-tree planner tries not to pay this price globally. It pays only for the interactions that the workload makes hard. Thus the polynomial oracle is the symbolic extreme of exactness, while the segment-tree model is the amortized practical compromise.

Third, the benchmark calibrates the preprocessing tradeoff. A future exact polylogarithmic structure must explain how it avoids both near-linear order recovery and explicit quadratic breakpoint compilation. The 3SUM reduction in Section 13 formalizes this tension: strongly sublinear exact operations are incompatible with truly subquadratic preprocessing under the stated hypothesis.

In this sense, the exact side has three layers. Radix-statistical aggregation is the unconditional linear word-RAM fallback. Pair-space is a compilation layer that spends near-quadratic resources to make selected Cross evaluations fast. The segment-tree interaction spine and fragmentation router are workload-sensitive acceleration layers that attempt to avoid invoking the linear fallback. The next section refines these accelerations by measuring which cross interactions are actually hard.

6 Fragmentation-Aware Exact Model

The epoch oracle is exact, but it hides the true structure of the query. Its $O(T^2)$ term comes from enumerating all pairs of offset-constant intervals. The following model refines this view.

6.1 Lazy Blocks, Dirty Elements, and Runs

Maintain a base array a_i^0 . The current value is represented as

$$x_i = a_i^0 + \tau_{b(i)} + \delta_i,$$

where $\tau_{b(i)}$ is a lazy block or node tag, and δ_i is a dirty correction caused by a partial update that has not yet been absorbed into the base structure.

A full-cover range translation of a clean object only changes its lazy tag. It does not change any internal pairwise distance:

$$|(x + \Delta) - (y + \Delta)| = |x - y|.$$

Partial updates create dirty elements or dirty subobjects. Dirty objects are handled by correction terms or local rebuilding.

For a query $Q = [l, r]$, the clean part is decomposed into shifted runs

$$(R_1, \tau_1), \dots, (R_g, \tau_g),$$

where each R_s is an index interval or canonical object on which all clean elements share the same offset τ_s . Let D_Q be the number of dirty elements in the query.

The exact query difficulty is controlled by several structural parameters:

- g : the number of offset runs;
- D_Q : the dirty mass in the query;
- $M = r - l + 1$: the raw query length;
- η_Q : value fragmentation in shifted-run value-CDQ;
- χ_Q : recursive cross-expansion size in index-CDQ;
- ω_Q : the number of hard overlap interactions after separation pruning.

6.2 Kinetic Cross Functions

For two clean canonical objects A, B , define

$$H_{A,B}(z) = \sum_{x \in A, y \in B} |y - x + z|, \quad z = \tau_B - \tau_A.$$

The raw values and their versions determine the function, while full-cover lazy translations only move the scalar evaluation point z . Let

$$d_1 \leq d_2 \leq \dots \leq d_p, \quad d_i \in D_{A,B} = \{y - x : x \in A, y \in B\},$$

and let $P_k = \sum_{i \leq k} d_i$, $P_0 = 0$, and $S = P_p$. For

$$k(z) = \#\{i : d_i \leq -z\},$$

the threshold formula becomes

$$H_{A,B}(z) = S - 2P_{k(z)} + (p - 2k(z))z.$$

A compiled interaction therefore stores the sorted breakpoints, prefix sums, and a cursor at the previous threshold rank. When the relative offset changes, exponential search from the cursor followed by binary search locates the new rank.

Route	Strength	Limitation
Ordinary lazy segment tree	handles interval tags	lacks value-order cross summaries
Comparison sorting	exact for arbitrary comparable values	$O(M \log M)$ per query
Radix-statistical fall-back	exact and linear for fixed-width integers	reads and reorders essentially all M values
Pair-space epoch oracle	compiled Cross tradeoff	quadratic preprocessing and coarse epochs
Segment-tree interaction spine	common amortized exact spine	cost depends on hard Cross residues
Spectral segment tree	fast algebraic route	approximate unless certified by an error bound

Table 1: Roles of the main routes. Radix aggregation is the direct exact word-RAM baseline; pair-space compilation buys fast Cross evaluation with quadratic preprocessing; the segment-tree interaction spine accelerates structured workloads.

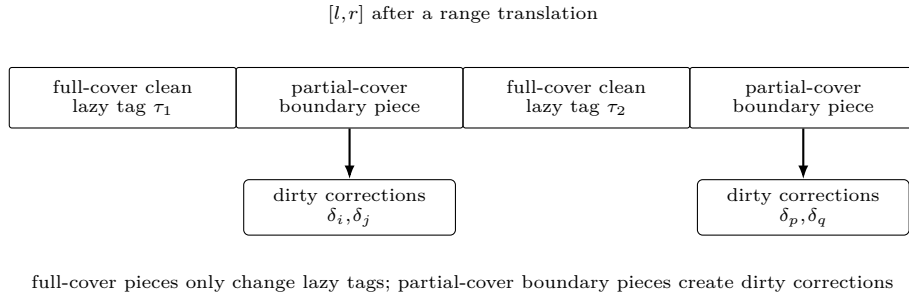
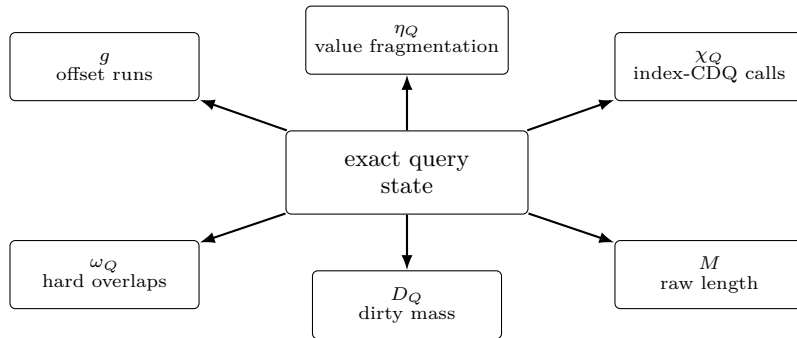


Figure 2: Lazy/dirty decomposition under a range translation. Full-cover canonical objects keep their internal pairwise answers unchanged; only boundary pieces become dirty.



the planner routes the query by the smallest structural obstruction

Figure 3: Exact-query structural parameters. The model separates difficulty into offset fragmentation, value fragmentation, CDQ expansion, overlap residue, dirty mass, and raw interval length.

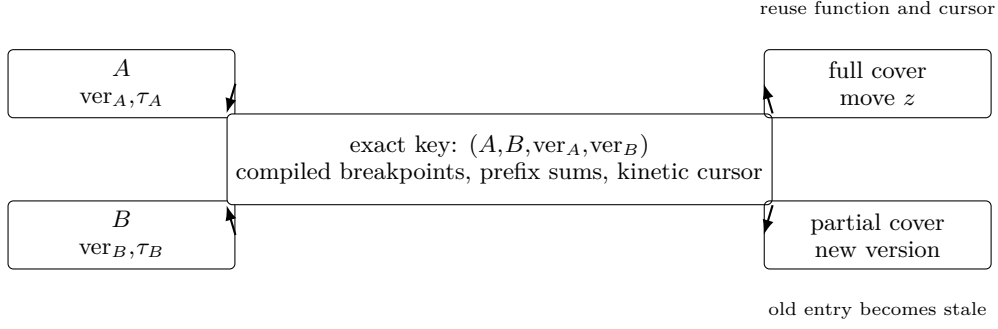


Figure 4: Kinetic versioned compilation. Full-cover translations move only the evaluation point of a compiled Cross function; partial changes invalidate the exact version key.

► **Theorem 16 (Kinetic compiled Cross evaluation).**

After $O(p \log p)$ comparison preprocessing, or linear preprocessing from an already ordered difference representation, a sequence $z_1, \dots, z_q$ of evaluations can be answered in

$$O\left(q + \sum_{t=2}^q \log(1 + |k(z_t) - k(z_{t-1})|)\right)$$

time by finger search. If the offset sequence is monotone, a one-directional cursor answers all evaluations in $O(q + p)$ total time.

Proof. Once the threshold rank is known, the prefix-sum formula is constant time. Galloping from the previous rank brackets a displacement of Δk in $O(1 + \log(1 + |\Delta k|))$ comparisons, after which binary search inside the bracket has the same order. For a monotone offset sequence the threshold rank is monotone in the opposite direction, so a linear cursor crosses each of the p breakpoints at most once. $\square$

This is stronger than treating the cache as a stateless hash lookup. Repeated full-cover translations preserve the compiled object and often move its query point only locally. A partial update or local rebuild increments the object version, making entries under the old exact key unreachable.

6.3 Sieve-Like Pruning and Interaction Routing

Memoization is most useful after an interaction has been classified. The first classification is a cheap value-geometry test. For a clean object A , let

$$I_A = [\min A + \tau_A, \max A + \tau_A]$$

be its shifted value interval, and define I_B analogously.

If the intervals are separated, for example

$$\max I_A \leq \min I_B,$$

then every shifted value in A is at most every shifted value in B . The exact cross term is therefore

$$\sum_{x \in A, y \in B} |y - x| = |A| \sum_{y \in B} y - |B| \sum_{x \in A} x.$$

The reverse separation case is symmetric. Thus separated interactions do not need pair-space Cross, value-CDQ,

or sorting. They are routed to a closed-form count/sum evaluator.

Let ω_Q denote the number of clean interactions in a query that remain hard after this separation pruning. Equivalently, ω_Q counts the overlap residue that still requires a heavy exact evaluator. The pruning does not discard any contribution; it only routes easy contributions away from the heavy machinery.

6.4 Rent-or-Buy Interaction Compilation

A global pair-space oracle pays for all interactions in advance, including those never queried. Versioned local compilation instead treats every structural pair $e = (A, B, \text{ver}_A, \text{ver}_B)$ as an independent online rent-or-buy decision. Let

$$\begin{aligned} C_e &= \text{cost of one cold exact evaluation,} \\ q_e &= \text{cost after compilation,} \\ B_e &= \text{compilation cost,} \end{aligned}$$

with $q_e < C_e$, and write $\delta_e = C_e - q_e$. The unavoidable baseline for k uses is kq_e ; each cold call rents for an additional δ_e , while compilation buys away all future premiums for B_e .

► **Theorem 17 (Two-competitive compilation threshold).** If $\delta_e > 0$, serve

$$r_e = \left\lceil \frac{B_e}{\delta_e} \right\rceil - 1$$

uses cold and compile immediately before the next use. For every number k of calls during the version lifetime, the resulting total cost is at most twice

$$\text{OPT}_e(k) = kq_e + \min\{k\delta_e, B_e\},$$

the better cost of never compiling and compiling before the first call. If $\delta_e \leq 0$, never compile.

Proof. For $k \leq r_e$, one has $k\delta_e < B_e$, so both the policy and the optimum rent every call. For $k > r_e$, the policy pays

$$kq_e + r_e\delta_e + B_e < kq_e + 2B_e.$$

Now $k \geq \lceil B_e/\delta_e \rceil$, so the optimum buys and equals $kq_e + B_e$. Hence the policy cost is less than twice the optimum. $\square$

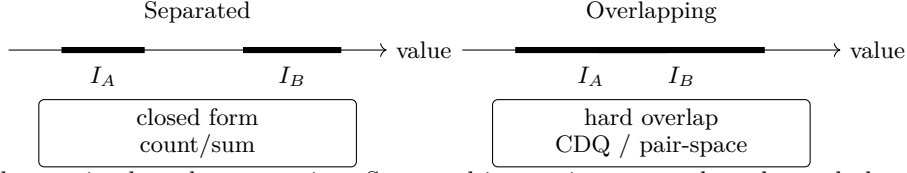


Figure 5: Sieve-like pruning by value separation. Separated interactions are evaluated exactly by count and sum; only overlapping interactions are routed to heavier evaluators.

The compiled object may be a sorted breakpoint function, a sparse Matrix-CDQ factorization, or another exact Cross representation. After compilation, the kinetic theorem above further reduces evaluation cost when relative offsets have locality. Version changes reset only the affected pair's rental history; no global quadratic structure is required.

The resulting amortization layer has three states:

cold exact evaluator $\rightarrow$ compiled Cross object
 $\rightarrow$ kinetic cursor evaluation

Hashing remains an implementation detail for locating the exact version key; the theoretical policy is determined by measured or certified values of B_e, C_e, q_e .

6.5 Dirty Correction by Multiset Regrouping

Let C be the genuinely clean multiset in the query, let

$$O_D = \{o_i : i \in D_Q\}$$

be the clean proxy values of the dirty elements, and let

$$V_D = \{v_i = o_i + \delta_i : i \in D_Q\}$$

be their true values. If the clean baseline includes the proxies, it equals $F(C \cup O_D)$, whereas the desired answer is $F(C \cup V_D)$. Expanding each aggregate into internal and cross pairs gives an exact regrouping identity.

► **Lemma 10 (Dirty regrouping identity).** *For arbitrary finite multisets C, O_D, V_D with $|O_D| = |V_D|$,*

$$F(C \cup V_D) = F(C \cup O_D) + F(V_D) - F(O_D) + \text{Cross}(C, V_D) - \text{Cross}(C, O_D).$$

Proof. Write $F(X \cup Y) = F(X) + F(Y) + \text{Cross}(X, Y)$ for disjointly labelled multisets and subtract the expansion with $Y = O_D$ from the one with $Y = V_D$. The clean internal term $F(C)$ cancels. No dirty-dirty pair is counted through a pointwise correction, so no secondary quadratic repair is needed. $\square$

► **Theorem 18 (Linear dirty residue for word integers).** *If a point-to-clean-range Cross query costs $R'(n)$, and $\text{Ord}(D_Q)$ denotes direct exact order recovery on the dirty values, dirty correction costs*

$$O(D_Q R'(n) + \text{Ord}(D_Q)).$$

For fixed-width integers this is $O(D_Q R'(n) + D_Q)$, removing the former D_Q^2 term. If the clean query histogram

and the two dirty histograms are already available in sorted-support form, both Cross terms and both internal sums can instead be evaluated by ordered scans in $O(d_C + d_O + d_V)$ time.

Proof. Compute $F(O_D)$ and $F(V_D)$ by direct order recovery on D_Q values. Evaluate each of the two dirty-clean Cross terms either by D_Q point-to-range calls or by merging their ordered supports with the clean histogram. Apply the regrouping lemma. $\square$

7 Shifted-Run Value-CDQ

The most promising exact improvement is to avoid enumerating the g^2 pairs of shifted runs. The idea is to divide by value rather than by run pair.

7.1 Static Range Value Count/Sum Oracle

Assume a static oracle on the base array that answers

$$\text{CntSum}(I, A, B) = \left(\#\{i \in I : A \leq a_i^0 \leq B\}, \sum_{i \in I, A \leq a_i^0 \leq B} a_i^0 \right)$$

in time $R(n)$. A merge-sort tree with prefix sums gives

$$R(n) = O(\log^2 n).$$

For integer compressed values, wavelet-tree-like structures may improve this factor.

7.2 Value-Domain Divide and Conquer

Consider a clean query decomposed into shifted runs

$$(R_1, \tau_1), \dots, (R_g, \tau_g).$$

For $i \in R_s$, the current value is

$$x_i = a_i^0 + \tau_s.$$

At a value-domain node $V = [L, R]$, split at M :

$$V_L = [L, M], \quad V_R = (M, R].$$

For each active run (I, τ) , the elements whose current values lie in V_L are exactly those satisfying

$$a_i^0 \in [L - \tau, M - \tau].$$

Thus CntSum returns their count and raw sum. The shifted sum is obtained by adding τ times the count.

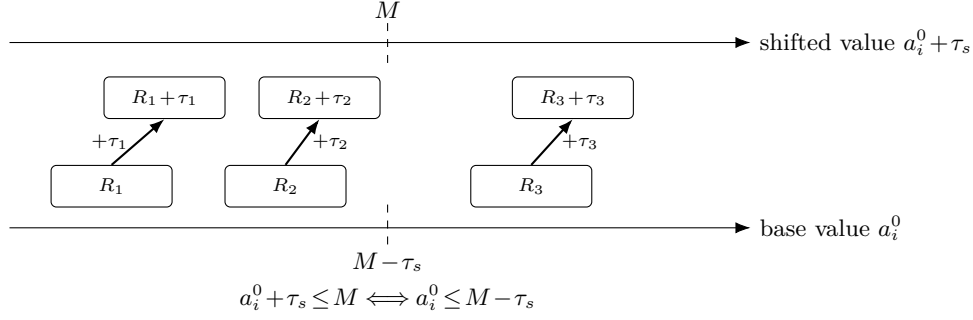


Figure 6: Shifted-run value-CDQ. Runs are shifted by lazy tags, while value splits are answered on base values by translating the threshold.

Let C_L, S_L and C_R, S_R be the total count and shifted sum of all active runs in the left and right value halves. All pairs separated by this value split contribute

$$C_L S_R - C_R S_L.$$

The algorithm recurses on the nonempty induced runs in V_L and V_R .

► **Lemma 11 (Unique value separation).** *Consider two clean elements with current values x and y in a value-CDQ recursion. If $x \neq y$, then there is a unique first recursion node at which the two values are sent to different children. If $x = y$, they are never separated and their pairwise contribution is zero.*

Proof. Both elements begin at the root value interval. As long as they are assigned to the same child, they remain together in the same recursive subproblem. Since the value intervals are nested and split into disjoint left and right halves, the first node at which the two values fall on different sides is unique. If no such node exists, the two values occupy the same final value atom, so their absolute difference is zero. $\square$

► **Lemma 12 (Split contribution).** *At a value-CDQ node with left count/sum (C_L, S_L) and right count/sum (C_R, S_R) , the total contribution of all pairs first separated at this node is*

$$C_L S_R - C_R S_L.$$

Proof. Every value in the left child is at most every value in the right child. Hence each separated pair contributes $y - x$, where x is from the left side and y is from the right side. Summing over all such pairs gives

$$\sum_{x \in L} \sum_{y \in R} (y - x) = C_L S_R - C_R S_L.$$

$\square$

► **Lemma 13 (Value-CDQ correctness).** *Shifted-run value-CDQ computes the exact clean pairwise L_1 sum.*

Proof. By the unique-separation lemma, every nonzero pair is charged at exactly one value split. By the split-contribution lemma, the contribution inserted at that split is exactly its absolute difference, aggregated over

all pairs first separated there. Therefore each nonzero pair contributes once and only once; equal-value pairs contribute zero and need not be charged. $\square$

Define

$$\eta_Q = \sum_{\nu} |\mathcal{A}_{\nu}|,$$

where ν ranges over the visited value-CDQ nodes and $\mathcal{A}_{\nu}$ is the set of active shifted runs at node ν .

► **Theorem 19 (Shifted-run value-CDQ).** *Given a clean query interval decomposed into shifted runs and a static CntSum oracle with query time $R(n)$, the exact clean one-dimensional pairwise L_1 aggregate can be computed in*

$$O(\eta_Q R(n))$$

time. The algorithm avoids the direct enumeration of g^2 run pairs.

Proof. At each value-CDQ node, the algorithm performs a constant number of CntSum calls for every active shifted run. Summing over all visited nodes gives $O(\eta_Q)$ oracle calls, each costing $R(n)$. Correctness follows from the value-CDQ correctness lemma: the recursion partitions all nonzero pair contributions by their first separating value split. $\square$

The parameter η_Q measures value fragmentation. It is different from g^2 , which measures offset fragmentation. When shifted runs are numerous but not highly fragmented in value space, η_Q can be much smaller than g^2 . In a low value-fragmentation regime, one may have

$$\eta_Q = O(g \log U)$$

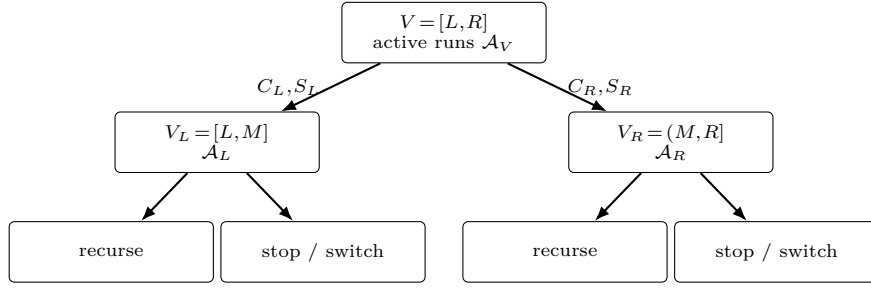
for a compressed value universe of size U , giving a clean query cost of

$$O(g \log U \cdot R(n)).$$

7.3 Truncated Value-CDQ

The value-CDQ recursion should not be forced to continue when another exact path is cheaper. At a node ν , let $a_{\nu} = |\mathcal{A}_{\nu}|$ be the number of active runs and let m_{ν} be the number of elements represented by the node. The implementation may choose the cheapest of:

$$a_{\nu}^2 P(n), \quad a_{\nu} R(n) \cdot h_{\nu}, \quad m_{\nu} \log m_{\nu},$$



each split contributes $C_L S_R - C_R S_L$; total oracle calls are measured by $\eta_Q = \sum_{\nu} |\mathcal{A}_{\nu}|$

Figure 7: Value-CDQ recursion tree. The quantity η_Q counts active shifted-run appearances over all visited value nodes.

where $P(n)$ is the cost of a pair-space Cross query and h_{ν} is the remaining value depth. These correspond respectively to run-pair Cross, continuing value-CDQ, and direct materialization/sorting.

This gives a recursive query planner rather than a single fixed algorithm.

8 Exact Hierarchical Factorization of the Absolute-Distance Kernel

The preceding value-domain recursion can be retained as a matrix factorization rather than immediately contracted to one scalar. This exposes the precise algebraic structure of the exact route and separates three issues that were previously conflated: the ordered geometry of the absolute-value kernel, the sparsity of the shifted-run supports, and the number of selectors that reuse the same interaction geometry.

8.1 Run-value matrix formulation

Fix g pairwise-disjoint shifted runs $R_1, \dots, R_g$, and let

$$v_1 < \dots < v_q$$

be the distinct current values appearing in their union. Define the run-value frequency matrix $H \in \mathbb{N}^{g \times q}$ and the ordered absolute-distance kernel $K \in \mathbb{R}^{q \times q}$ by

$$H_{sp} = \#\{i \in R_s : x_i = v_p\}, \quad K_{pq} = |v_p - v_q|.$$

The symmetric run interaction matrix $C \in \mathbb{R}^{g \times g}$ is

$$C_{st} = \sum_{i \in R_s} \sum_{j \in R_t} |x_i - x_j|.$$

Thus C_{ss} counts each unordered internal pair twice, whereas C_{st} for $s \neq t$ is the ordinary cross interaction between two runs.

► **Lemma 14 (Kernel product identity).** *The run interaction matrix satisfies*

$$C = H K H^{\top}.$$

Consequently, for a binary selector $z \in \{0, 1\}^g$, the total pairwise L_1 sum inside the union of the selected runs is

$$F(z) = \frac{1}{2} z^{\top} C z.$$

Proof. The (s, t) -entry of $H K H^{\top}$ is

$$\sum_{p=1}^q \sum_{r=1}^q H_{sp} |v_p - v_r| H_{tr},$$

which groups the pairs $(i, j) \in R_s \times R_t$ by their two values and is exactly C_{st} . In $z^{\top} C z$, every cross-run unordered pair appears twice by symmetry, and every internal unordered pair appears twice in the diagonal entry of its run. Division by two therefore gives the selected-union answer. $\square$

8.2 Separated kernel blocks have exact rank two

The matrix identity is useful because ordering makes the absolute-distance kernel exactly low rank away from its diagonal.

► **Lemma 15 (Rank-two separated block).** *Let $I, J \subseteq \{1, \dots, q\}$ satisfy $p < r$ for every $p \in I$ and $r \in J$. Writing $v_I = (v_p)_{p \in I}$, $v_J = (v_r)_{r \in J}$, one has*

$$K_{I,J} = \mathbf{1}_I v_J^{\top} - v_I \mathbf{1}_J^{\top}.$$

In particular,

$$\text{rank}(K_{I,J}) \leq 2.$$

Proof. For every $p \in I$ and $r \in J$, ordering gives $|v_p - v_r| = v_r - v_p$. The displayed outer-product formula has exactly this (p, r) -entry and is a sum of two rank-one matrices. $\square$

This lemma is the matrix form of separation pruning. The closed count/sum formula is not an unrelated shortcut: it is the contraction of an exact rank-two kernel block.

8.3 Binary value-tree factorization

Let $\mathcal{T}$ be any binary tree whose leaves, from left to right, are the ordered value atoms $v_1, \dots, v_q$. For an internal node ν , let L_{ν} and R_{ν} denote the value atoms in its left and right children. Define four vectors in $\mathbb{R}^g$:

$$\begin{aligned} (c_{\nu,L})_s &= \sum_{p \in L_{\nu}} H_{sp}, & (p_{\nu,L})_s &= \sum_{p \in L_{\nu}} v_p H_{sp}, \\ (c_{\nu,R})_s &= \sum_{p \in R_{\nu}} H_{sp}, & (p_{\nu,R})_s &= \sum_{p \in R_{\nu}} v_p H_{sp}. \end{aligned}$$

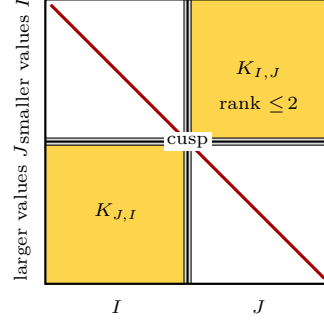


Figure 8: Ordered absolute-distance kernel. Blocks separated from the cusp diagonal are exactly rank two; only blocks intersecting the diagonal require further value refinement.

The local run-interaction factor at ν is

$$B_\nu = c_{\nu,L} p_{\nu,R}^\top - p_{\nu,L} c_{\nu,R}^\top + p_{\nu,R} c_{\nu,L}^\top - c_{\nu,R} p_{\nu,L}^\top. \quad (2)$$

► **Theorem 20 (Exact hierarchical factorization).** *The run interaction matrix has the exact decomposition*

$$C = \sum_{\nu \in \text{Int}(\mathcal{T})} B_\nu, \quad \text{rank}(B_\nu) \leq 4.$$

Proof. Each ordered pair of distinct value atoms (v_p, v_r) has a unique lowest value-tree node ν whose two children separate the atoms. Assume $v_p \in L_\nu$ and $v_r \in R_\nu$. The first two outer products in (2) contribute $v_r - v_p$ to the interaction from a run containing v_p to a run containing v_r ; the last two contribute the symmetric orientation. Thus all ordered element pairs with these two values are charged at ν , and they are charged nowhere else. Equal-value pairs contribute zero and require no factor. Summing over all internal nodes gives every entry of C . Equation (2) is a sum of four rank-one matrices, so each factor has rank at most four. $\square$

► **Corollary 2 (Scalar value-CDQ as matrix contraction).** *Let*

$$\begin{aligned} C_{\nu,L} &= \mathbf{1}^\top c_{\nu,L}, & S_{\nu,L} &= \mathbf{1}^\top p_{\nu,L}, \\ C_{\nu,R} &= \mathbf{1}^\top c_{\nu,R}, & S_{\nu,R} &= \mathbf{1}^\top p_{\nu,R}. \end{aligned}$$

Then

$$\frac{1}{2} \mathbf{1}^\top B_\nu \mathbf{1} = C_{\nu,L} S_{\nu,R} - S_{\nu,L} C_{\nu,R}.$$

Hence the scalar shifted-run value-CDQ algorithm is exactly the contraction of the hierarchical matrix factors against the all-ones selector.

Proof. Expanding the four outer products gives two equal copies of $C_{\nu,L} S_{\nu,R} - S_{\nu,L} C_{\nu,R}$; the factor $1/2$ removes the duplication caused by the symmetric interaction matrix. $\square$

8.4 Sparse factor size and construction time

Let $\mathcal{A}_\nu$ be the set of runs with at least one value in the value interval of ν . Let $\mathcal{V}_Q$ be the internal nodes visited by the value recursion, including routing nodes whose factor vanishes because only one child is occupied, and

let h_Q be the number of visited nodes whose factor is nonzero. Define the active-incidence complexity

$$\eta_Q = \sum_{\nu \in \mathcal{V}_Q} |\mathcal{A}_\nu|.$$

The four factor vectors are sparse outside $\mathcal{A}_\nu$; zero factors are not stored. If the nonzero factor columns are concatenated as

$$\begin{aligned} L_C &= [c_{\nu,L}]_\nu, & L_P &= [p_{\nu,L}]_\nu, \\ R_C &= [c_{\nu,R}]_\nu, & R_P &= [p_{\nu,R}]_\nu, \end{aligned}$$

then

$$C = L_C R_P^\top - L_P R_C^\top + R_P L_C^\top - R_C L_P^\top, \quad (3)$$

and the total number of stored nonzero scalar entries in the four factor matrices is $O(\eta_Q)$. The build parameter therefore counts traversal work, while h_Q counts the retained low-rank factors.

► **Theorem 21 (Factor construction complexity).** *Assume that, for one shifted run and one value interval, count and shifted-value-sum statistics can be obtained in $R(n)$ time. Then the factorization (3) can be constructed in*

$$O(\eta_Q R(n))$$

time and stored in $O(\eta_Q)$ space.

Proof. At every visited value-tree node, each active run requires a constant number of count/sum statistics to determine its left and right entries; one child can also be obtained by subtracting from the parent statistic. Thus the number of oracle calls is $O(\eta_Q)$. The same active incidences bound the total number of nonzero vector entries retained in the four factor matrices. $\square$

For canonical segment-tree runs that store their own sorted base values and prefix sums, one translated threshold is answered by binary search in $O(\log n)$ time. For an arbitrary index interval represented through a merge-sort tree, the generic bound remains $R(n) = O(\log^2 n)$.

For a fixed-width integer implementation, $\mathcal{T}$ may be a dyadic tree over the signed value universe. Empty branches are never materialized, and unary routing paths are counted in η_Q . Conceptually suppressing those empty paths yields the compressed tree on the distinct current values used in the kernel formulation above.

The active-incidence parameter also admits a support bound that is known before the numeric factors are materialized. Suppose the ordered integer universe is represented by a balanced binary tree with U leaves, and run s contains d_s distinct current values.

► **Lemma 16 (Support-entropy bound).** *For a balanced binary value tree,*

$$\eta_Q = O\left(\sum_{s=1}^g d_s \left(1 + \log \frac{U}{d_s}\right)\right).$$

The same bound holds for every pruned subrecursion of the value tree.

Proof. At depth j , the support of run s can occupy at most $\min\{2^j, d_s\}$ value nodes. Summing this quantity over the $\log U + 1$ depths gives

$$\sum_{j=0}^{\log U} \min\{2^j, d_s\} = O\left(d_s + d_s \log \frac{U}{d_s}\right).$$

Summing over the runs proves the claim. Pruning can only remove occupied run–node incidences. $\square$

8.5 Interaction-only pruning

If each run already stores its internal unordered pairwise sum

$$P_s = \sum_{i < j, i, j \in R_s} |x_i - x_j|,$$

then the hierarchical recursion need only retain interactions between different runs. Let

$$D_\nu = \text{Diag}(\text{diag } B_\nu).$$

The zero-diagonal cross-run matrix is

$$C^\times = C - \text{Diag}(\text{diag } C) = \sum_{\nu} (B_\nu - D_\nu).$$

For a binary selector z ,

$$F(z) = \sum_{s=1}^g z_s P_s + \frac{1}{2} \sum_{\nu} (z^\top B_\nu z - \text{diag}(B_\nu)^\top z). \quad (4)$$

A value node containing values from at most one active run contributes only to the removed diagonal and can be terminated immediately. Let $\zeta_Q \leq \eta_Q$ denote the active-incidence count in this interaction-only recursion.

► **Corollary 3 (Interaction-only exact query).** *When run-internal sums are available, the exact clean query can be evaluated in*

$$O(\zeta_Q R(n) + g)$$

time with $O(\zeta_Q + g)$ temporary space. In particular, a canonical-node implementation with sorted node lists has cost $O(\zeta_Q \log n + g)$.

Proof. Equation (4) separates stored run-internal contributions from cross-run factors. Nodes with only one active run have zero off-diagonal contribution and are not expanded. Every remaining active incidence performs only a constant number of range-statistic operations and stores a constant number of scalars. $\square$

8.6 Batch selectors and rectangular matrix multiplication

The matrix form becomes algorithmically distinct from scalar CDQ when many selectors reuse one fixed shifted-run geometry. Let $Y \in \{0,1\}^{B \times g}$ contain B run selectors as rows. Define

$$A = Y L_C, \quad E = Y R_P, \quad D = Y L_P, \quad G = Y R_C.$$

For selector row b , the pairwise answer is

$$F(Y_{b,*}) = \sum_{\nu=1}^{h_Q} (A_{b\nu} E_{b\nu} - D_{b\nu} G_{b\nu}). \quad (5)$$

Here $\text{MM}(a,b,c)$ denotes the time to multiply an $a \times b$ matrix by a $b \times c$ matrix.

► **Theorem 22 (Batch selector evaluation).** *After constructing the full factorization (3), B subset queries over the same g shifted runs can be answered in*

$$O(\min\{B\eta_Q, \text{MM}(B, g, h_Q) + Bh_Q\})$$

time, in addition to the cost of reading the selector matrix.

Proof. The sparse route evaluates the four selector–factor products by scanning the $O(\eta_Q)$ nonzeros once per selector, followed by $O(Bh_Q)$ elementwise products and row sums. Alternatively, compute the four products in (5) by rectangular matrix multiplication; the constant number of products is absorbed by $\text{MM}(B, g, h_Q)$, and the final Hadamard products and row reductions cost $O(Bh_Q)$. $\square$

For one query, matrix notation does not eliminate the active-incidence work: the scalar contraction is already optimal for the constructed factors. The matrix-multiplication route becomes relevant only when a fixed run configuration is reused by many selectors, for example inside an epoch, an offline batch, or a memoized structural interaction. Maintaining the factorization itself under arbitrary run translations remains a separate dynamic problem.

9 Lazy-Incremental Curvature and Histogram Transforms

The hierarchical factorization above is exact, but a factorization tied to the current shifted values may have to be rebuilt after a run moves. This section isolates a complementary representation in which translations act algebraically before any value-tree decomposition is chosen. The central observation is that the absolute-distance interaction has three equivalent forms:

$$\begin{aligned} \text{value histogram} &\longleftrightarrow \text{difference correlation} \\ &\longleftrightarrow \text{threshold-feature quadratic form.} \end{aligned}$$

A full translation is respectively a monomial multiplication, a shift of the correlation, and an affine isometry. A partial translation produces a two-term histogram delta. These identities do not by themselves yield a worst-case polylogarithmic exact structure—the 3SUM barrier remains—but they identify the exact algebra needed for lazy tags, incremental corrections, and periodic Matrix-CDQ compaction.

9.1 Laurent histograms and monomial translations

For an integer-valued finite multiset A , let $h_A(x)$ denote the multiplicity of x . Its Laurent histogram polynomial is

$$P_A(z) = \sum_{x \in \mathbb{Z}} h_A(x) z^x.$$

Only finitely many coefficients are nonzero. If S is a submultiset of A , write $A \setminus S$ for multiset subtraction.

► **Lemma 17 (Monomial action of translation).** *For every integer Δ ,*

$$P_{A+\Delta}(z) = z^\Delta P_A(z).$$

More generally, if only a submultiset $S \subseteq A$ is translated by Δ , then

$$P_{(A \setminus S) \uplus (S + \Delta)}(z) = P_A(z) + (z^\Delta - 1)P_S(z). \quad (1)$$

Proof. Every occurrence of a value x contributes z^x . Translating it to $x + \Delta$ replaces that monomial by $z^{x+\Delta} = z^\Delta z^x$. Summing over all occurrences proves the first identity. For the second, subtract the old contribution P_S and add its shifted copy $z^\Delta P_S$. $\square$

Equation (1) is the basic incremental identity. A full-cover update requires only one lazy exponent. A partial update does not create an arbitrary new distribution: it creates the signed pair of components

$$-P_S(z), \quad z^\Delta P_S(z).$$

The unresolved algorithmic issue is how compactly the affected subhistogram P_S and its interactions with the rest of the structure can be represented.

9.2 Curvature–correlation transform

For two multisets A, B , define the difference–correlation polynomial

$$Q_{A,B}(z) = P_A(z^{-1})P_B(z) = \sum_{d \in \mathbb{Z}} q_d z^d.$$

Its coefficient has the combinatorial meaning

$$q_d = \#\{(a, b) \in A \times B : b - a = d\}.$$

Define the translated cross function

$$H_{A,B}(t) = \sum_{a \in A, b \in B} |b - a + t| = \sum_d q_d |d + t|.$$

► **Theorem 23 (Curvature–correlation transform).** *For every integer t ,*

$$H_{A,B}(t+1) - 2H_{A,B}(t) + H_{A,B}(t-1) = 2q_{-t}. \quad (2)$$

Moreover, if A and B are translated by α and β , respectively, then

$$Q_{A+\alpha, B+\beta}(z) = z^{\beta-\alpha} Q_{A,B}(z), \quad (3)$$

$$H_{A+\alpha, B+\beta}(t) = H_{A,B}(t + \beta - \alpha). \quad (4)$$

Proof. For fixed d , the discrete second difference of $|d+t|$ is two when $d+t=0$ and zero otherwise. Summing with multiplicities q_d gives (2). By Lemma 17,

$$P_{A+\alpha}(z^{-1})P_{B+\beta}(z) = z^{-\alpha} P_A(z^{-1})z^\beta P_B(z),$$

which proves (3). The second equality follows directly from

$$|(b+\beta) - (a+\alpha) + t| = |b - a + t + \beta - \alpha|.$$

$\square$

Thus the entire exact cross function is a twice-discrete integral of the pair-difference correlation. In the lower-bound section, the same identity extracts a prescribed pair-sum multiplicity. Here it is used in the opposite direction: the correlation is an exact state representation on which relative translations act only by a shift.

A partial translation also has an exact incremental rule.

► **Corollary 4 (Incremental correlation update).** *Let $S \subseteq A$, and let*

$$A' = (A \setminus S) \uplus (S + \Delta).$$

Then

$$Q_{A',B}(z) = Q_{A,B}(z) + (z^{-\Delta} - 1)P_S(z^{-1})P_B(z). \quad (5)$$

If both sides receive histogram deltas δP_A and δP_B , then the new correlation is the old correlation plus the two linear cross terms and the bilinear delta–delta term.

Proof. Substitute (1) into the first factor of $P_A(z^{-1})P_B(z)$. The two-sided statement is the ordinary bilinear expansion

$$(P_A + \delta P_A)(z^{-1})(P_B + \delta P_B)(z).$$

$\square$

9.3 Exact threshold–feature embedding

The same kernel also admits an exact Hilbert-space representation. For an integer x , define the finitely supported oriented path feature $\psi_x: \mathbb{Z} \rightarrow \{-1, 0, 1\}$ by

$$\psi_x(t) = \begin{cases} 1, & 0 \leq t < x, \\ -1, & x \leq t < 0, \\ 0, & \text{otherwise.} \end{cases}$$

The first case is empty when $x \leq 0$, and the second is empty when $x \geq 0$. Equip finitely supported sequences with the standard inner product.

► **Lemma 18 (Exact threshold embedding).** *For all integers x, y ,*

$$\|\psi_x - \psi_y\|_2^2 = |x - y|.$$

Consequently, for a multiset $A = \{x_1, \dots, x_m\}$, if

$$S_A = \sum_{i=1}^m \psi_{x_i}, \quad E_A = \sum_{i=1}^m \|\psi_{x_i}\|_2^2,$$

then

$$\sum_{1 \leq i < j \leq m} |x_i - x_j| = mE_A - \|S_A\|_2^2. \quad (6)$$

Proof. The difference $\psi_x - \psi_y$ is the oriented indicator of the integer interval between x and y . It therefore has exactly $|x - y|$ nonzero coordinates, each of squared magnitude one. The second identity is the standard pairwise squared-distance formula

$$\sum_{i < j} \|u_i - u_j\|_2^2 = m \sum_i \|u_i\|_2^2 - \left\| \sum_i u_i \right\|_2^2$$

applied to $u_i = \psi_{x_i}$. $\square$

This exact embedding should be contrasted with the finite-dimensional spectral approximation later in the paper. The threshold feature has unbounded ambient dimension but finite support on every bounded instance; no kernel approximation has occurred.

Let T_Δ be the shift operator

$$(T_\Delta f)(t) = f(t - \Delta).$$

► **Lemma 19 (Translation cocycle).** *For all integers x, Δ ,*

$$\psi_{x+\Delta} = \psi_\Delta + T_\Delta \psi_x, \quad (7)$$

$$\psi_{x+\Delta} - \psi_x = T_\Delta \psi_\Delta. \quad (8)$$

Hence translation acts as an affine isometry on the exact feature space.

Proof. Each identity expresses concatenation of oriented integer paths. The path from 0 to $x + \Delta$ is the path from 0 to Δ followed by the translate by Δ of the path from 0 to x , with opposite traversals canceling when the two paths overlap. Equivalently, the increment from the path ending at x to the path ending at $x + \Delta$ is the translate by x of the path from 0 to Δ . Shift operators preserve the inner product, so the linear part is an isometry. $\square$

For a whole multiset,

$$S_{A+\Delta} = |A| \psi_\Delta + T_\Delta S_A.$$

Equation (6) is invariant under this affine action, recovering the elementary fact that a common translation preserves all internal distances.

9.4 Partial translations, box convolution, and tent kernels

Let S be the submultiset translated by Δ . By (8), the change in its aggregate threshold feature is

$$\delta S = \sum_x h_S(x) T_x \psi_\Delta = h_S * \psi_\Delta, \quad (9)$$

where the last expression is discrete convolution. Thus a partial range translation becomes convolution of the affected value histogram with an oriented box of width $|\Delta|$.

► **Lemma 20 (Tent-kernel quadratic correction).** *For all integers x, y, Δ ,*

$$\langle T_x \psi_\Delta, T_y \psi_\Delta \rangle = (|\Delta| - |x - y|)_+,$$

where $(r)_+ = \max\{r, 0\}$. Therefore

$$\|\delta S\|_2^2 = \sum_{x, y} h_S(x) h_S(y) (|\Delta| - |x - y|)_+. \quad (10)$$

Proof. The vector $T_x \psi_\Delta$ is the signed indicator of an integer interval of length $|\Delta|$, translated to start at x in the oriented sense. Two such vectors have the same sign on their overlap. Their inner product is therefore the overlap length, which is $(|\Delta| - |x - y|)_+$. Expanding the squared norm of (9) proves (10). $\square$

The tent kernel is compactly supported and piecewise affine. In particular, interactions between affected values farther than $|\Delta|$ apart vanish from the quadratic correction, while separated ordered blocks inside the support still admit rank-two count/sum formulas. The incremental kernel is therefore at least as compatible with hierarchical pruning as the original absolute-distance kernel.

A direct multiset interpretation is also useful. If $A = S \uplus R$, then translating S does not change its internal pairwise sum, and

$$F((S + \Delta) \uplus R) - F(S \uplus R) = H_{S, R}(\Delta) - H_{S, R}(0). \quad (11)$$

Thus all nontrivial work of a partial translation is an incremental cross interaction between the moved and unmoved parts.

9.5 Node-local lazy coordinates

Consider an index segment-tree node u with children L, R . Store each child histogram in local coordinates, with scalar tags τ_L, τ_R . The current child histograms are $z^{\tau_L} P_L(z)$ and $z^{\tau_R} P_R(z)$. Let

$$Q_{L, R}(z) = P_L(z^{-1}) P_R(z)$$

be their residual correlation, and let

$$\lambda_u = \tau_R - \tau_L.$$

Then the exact internal answer satisfies

$$F_u = F_L + F_R + H_{L, R}(\lambda_u). \quad (12)$$

A full-cover update of u adds the same value to both children. It changes only a common ancestor tag: λ_u , $Q_{L, R}$, and F_u remain unchanged. If a partial update changes only the left residual histogram by δP_L , then

$$Q'_{L, R}(z) = Q_{L, R}(z) + \delta P_L(z^{-1}) P_R(z), \quad (13)$$

with the symmetric formula for a right-side change. Hence partial updates propagate structured correlation deltas upward, while full-cover updates remain purely lazy.

► **Theorem 24 (Exact lazy-incremental representation).** *Suppose a dynamic segment-tree node is represented by local histogram polynomials, scalar translation tags, internal pairwise values, and residual child correlations. Then:*

1. a full-cover translation changes only one scalar tag and leaves every residual histogram, residual correlation, and internal pairwise value unchanged;
2. a partial translation of a submultiset S contributes the two-term histogram delta $(z^\Delta - 1)P_S(z)$;
3. every affected ancestor correlation changes by a bilinear product between that delta and the unchanged sibling histogram, plus a delta-delta term if both sides changed;
4. the exact node answer is recovered by evaluating the residual cross function at the current relative tag as in (12).

Proof. The first claim follows from common-translation invariance and Equation (4). The second is Lemma 17. The third is Corollary 4. The fourth is the decomposition of unordered pairs into left-internal, right-internal, and left-right classes. $\square$

The theorem is representational: it proves that the dynamic change is algebraically sparse. It does not assert that the affected subhistogram or every resulting correlation can always be materialized in polylogarithmic time. Adversarial instances may still expose near-linear support or quadratically many breakpoints.

9.6 Buffered components and Matrix-CDQ compaction

The two-term delta suggests keeping a compiled base plus a short log of symbolic components. Abstractly, write a local histogram as

$$P(z) = B(z) + \sum_{r=1}^k \varepsilon_r z^{\lambda_r} G_r(z), \quad \varepsilon_r \in \{-1, +1\}. \quad (14)$$

Each partial translation appends a negative old component and a positive shifted component. Sharing the underlying G_r in a directed acyclic representation prevents immediate duplication of its support.

For a histogram vector h , let K be the absolute-distance kernel. Its internal pairwise sum is

$$\mathcal{F}(h) = \frac{1}{2} h^\top K h.$$

Substituting (14) expands $\mathcal{F}$ into the base value, base-component interactions, and the Gram matrix of component interactions. The exact hierarchical factorization from Section 8 can be used as a compactor for these bilinear terms: separated component blocks are rank two, value splits generate rank-at-most-four factors, and repeated component selectors can be batched.

The following generic buffering statement separates the algebra from the choice of correlation oracle.

► **Theorem 25 (Local buffered-compaction bound).** *At a fixed structural node, suppose:*

- inserting one new symbolic component and accounting for its interaction with the compiled base costs at most C_u , and accounting for its interaction with each existing pending component also costs at most C_u ;

- compacting at most T pending components into the compiled base costs at most B_u .

If the node is compacted whenever its pending list reaches T , then the amortized local cost per inserted component is

$$O\left(TC_u + \frac{B_u}{T}\right).$$

Choosing $T = \Theta(\sqrt{B_u/C_u})$, when this value is admissible, gives

$$O(\sqrt{B_u C_u})$$

amortized local cost.

Proof. Before compaction, a newly inserted component interacts with the compiled base and with at most $T-1$ pending components, costing $O(TC_u)$. One compaction of cost B_u is charged to the T insertions that filled the buffer, contributing $O(B_u/T)$ per insertion. Balancing the two terms yields the stated choice of T . $\square$

For a range update, full-cover canonical nodes incur only lazy-tag changes. Only the $O(\log n)$ partial-cover boundary nodes can append new local components. If node u uses threshold T_u , the resulting conditional amortized update bound is

$$O\left(\log n + \sum_{u \in \partial I} \left(T_u C_u + \frac{B_u}{T_u}\right)\right),$$

where ∂I is the set of partially covered structural nodes. This formula is not yet a uniform worst-case improvement: its strength depends on proving nontrivial bounds for C_u , B_u , component sharing, and the number of configurations reused before invalidation. It does, however, replace the informal instruction “rebuild when dirty” by an exact delta representation and a concrete amortization target.

9.7 Relation to the exact and approximate frontiers

The three representations can be summarized as follows:

$$\begin{aligned} P_A(z) &\mapsto z^\Delta P_A(z) && \text{(monomial action),} \\ Q_{A,B}(z) &\mapsto z^{\beta-\alpha} Q_{A,B}(z) && \text{(relative shift),} \\ S_A &\mapsto |A| \psi_\Delta + T_\Delta S_A && \text{(affine isometry).} \end{aligned}$$

All three are exact. Matrix-CDQ supplies a sparse hierarchical realization when the histogram or correlation has compressible ordered structure. The spectral route later replaces the infinite exact threshold feature by a finite-dimensional approximate representation, in which the same translation action becomes phase rotation.

The curvature identity therefore has a dual role. Algorithmically, it converts exact cross interactions into shiftable pair-difference correlations. In Section 13, it shows that recovering those correlations at arbitrary points is powerful enough to encode Tripartite 3SUM. The transformation explains both why lazy incrementality is possible on structured states and why a generic exact polylogarithmic implementation should not be expected without paying elsewhere in preprocessing, support size, or approximation error.

10 A Concrete Shift–Meld Histogram Segment Tree

The previous section gives an exact algebraic update law, but leaves open how a histogram is represented when one wants a complete data structure rather than a symbolic component model. This section gives a direct materialized instantiation. Every index-segment-tree node stores its current value histogram in a persistent join-based ordered map. The map supports a lazy shift of all keys, combines equal-key multiplicities, and maintains the exact pairwise absolute-difference sum as an augmented field. Full-cover range additions are therefore constant-time changes at each canonical node; only partially covered ancestors rebuild their histograms by melding the two child maps.

This structure is not claimed to beat the linear radix fallback in the adversarial case. Its purpose is sharper: it supplies an end-to-end exact implementation whose update and query costs are controlled by the numbers of distinct values in the structural pieces, thereby instantiating the lazy-incremental transform without leaving the local rebuild cost abstract.

10.1 Weighted ordered histogram summaries

For a finite histogram $h: \mathbb{Z} \rightarrow \mathbb{Z}_{\geq 0}$, define

$$\begin{aligned} d(h) &= |\text{supp } h|, \\ N(h) &= \sum_x h(x), \\ W(h) &= \sum_x xh(x), \\ \mathcal{F}(h) &= \sum_{x < y} h(x)h(y)(y-x). \end{aligned}$$

The quantity $\mathcal{F}(h)$ is exactly the unordered pairwise L_1 sum of the represented multiset; equal-value pairs contribute zero.

► **Lemma 21 (Ordered histogram merge).** *Let a and b be two histograms such that every key in $\text{supp } a$ is smaller than every key in $\text{supp } b$. Then*

$$\mathcal{F}(a+b) = \mathcal{F}(a) + \mathcal{F}(b) + N(a)W(b) - N(b)W(a). \quad (15)$$

The other three summaries satisfy

$$\begin{aligned} d(a+b) &= d(a) + d(b), & N(a+b) &= N(a) + N(b), \\ W(a+b) &= W(a) + W(b). \end{aligned}$$

Proof. The internal pairs of a and b contribute $\mathcal{F}(a)$ and $\mathcal{F}(b)$. Every cross pair has the form $x < y$ and contributes $y-x$. Hence the total cross contribution is

$$\sum_{x,y} a(x)b(y)(y-x) = N(a)W(b) - N(b)W(a).$$

The remaining identities are immediate. $\square$

A balanced search-tree node stores one key x with multiplicity c_x , its left and right subtrees, and the four summaries above. Since all left keys are smaller than x and all right keys are larger, Lemma 21 recomputes every

augmented field in constant time after a rotation, split, join, or path-copy operation. Multiplicities of equal keys are combined at one node.

► **Lemma 22 (Lazy shift of a histogram).** *Let $h^{+\Delta}$ be obtained by replacing every key x of h by $x+\Delta$. Then*

$$\begin{aligned} d(h^{+\Delta}) &= d(h), & N(h^{+\Delta}) &= N(h), \\ W(h^{+\Delta}) &= W(h) + \Delta N(h), & \mathcal{F}(h^{+\Delta}) &= \mathcal{F}(h). \end{aligned}$$

Thus a root lazy-key tag implements a common translation in $O(1)$ time.

Proof. Translation preserves key equality and every pairwise difference. The weighted sum gains one copy of Δ for each represented element. $\square$

10.2 Join-based multiset maps

Let $\text{Union}_+(A, B)$ denote ordered-map union in which multiplicities at an equal key are added. Join-based balanced trees support union, intersection, and difference on ordered sets of sizes $a \leq b$ in

$$O\left(a \log \left(\frac{b}{a} + 1\right)\right)$$

work [7]. The same recursion applies to weighted maps: the split key is processed once, equal-key payloads are combined, and the augmented summaries are recomputed by Lemma 21. Define

$$\mathcal{J}(a, b) = \begin{cases} 0, & \min\{a, b\} = 0, \\ \min\{a, b\} \log \left(1 + \frac{\max\{a, b\}}{\min\{a, b\}}\right), & \text{otherwise.} \end{cases} \quad (16)$$

► **Lemma 23 (Weighted persistent union).** *Two weighted histogram maps with a and b distinct keys can be combined by Union_+ in $O(\mathcal{J}(a, b))$ work. With path copying, both inputs remain available and the number of newly allocated map nodes is within the same asymptotic bound. A lazy shift of either input is preserved through split and join by pushing its root tag only along copied paths.*

Proof. Apply the join-based union recursion to the keys, combining multiplicities when the same key occurs in both maps. The work analysis is unchanged because payload arithmetic and summary recomputation are constant-time operations per visited search-tree node. Path copying creates one immutable replacement for each visited node and does not visit additional nodes. Lazy tags are pushed only when a visited node is exposed. $\square$

The persistence here is an implementation device, not a requirement to retain historical array versions. Once a range operation finishes, reference counting or tracing garbage collection may reclaim roots no longer reachable from the current index segment tree.

Algorithm 1 Range addition in the shift-meld structure

```

1: ADD( $u, L, R, \Delta$ )
2: if  $I_u \subseteq [L, R]$  then
3:   lazily shift all keys of  $T_u$  by  $\Delta$ ;  $\tau_u \leftarrow \tau_u + \Delta$ 
4:   return
5: end if
6: push  $\tau_u$  to the children and reset  $\tau_u \leftarrow 0$ 
7: if  $I_{u_L} \cap [L, R] \neq \emptyset$  then
8:   ADD( $u_L, L, R, \Delta$ )
9: end if
10: if  $I_{u_R} \cap [L, R] \neq \emptyset$  then
11:   ADD( $u_R, L, R, \Delta$ )
12: end if
13:  $T_u \leftarrow \text{Union}_+(T_{u_L}, T_{u_R})$ 

```

10.3 The shift-meld range structure

Build a balanced segment tree over the index domain. Each index node u stores:

- a persistent weighted histogram map T_u containing the current values in its index interval;
- a scalar pending range-add tag τ_u for its children;
- the summaries d_u, N_u, W_u, F_u exposed by the root of T_u .

Here $F_u = \mathcal{F}(T_u)$ is the exact internal answer for the node interval.

For a full-cover update of u by Δ , clone the map root, add Δ to its lazy-key tag, update $W_u \leftarrow W_u + \Delta N_u$, and set $\tau_u \leftarrow \tau_u + \Delta$. By Lemma 22, F_u is unchanged. For a partial update, first push τ_u to the two children, recurse into the intersected children, and then perform the pull

$$T_u = \text{Union}_+(T_{u_L}, T_{u_R}). \quad (17)$$

All node summaries, including the exact pairwise answer, are read from the resulting map root.

Let ∂I be the set of index-tree nodes visited by an update interval I but not fully covered by it. Only these nodes execute the meld pull (17).

► **Theorem 26 (Support-sensitive exact range addition).** *The shift-meld histogram segment tree supports an exact range addition on I in*

$$O\left(\log n + \sum_{u \in \partial I} \mathcal{J}(d_{u_L}, d_{u_R})\right) \quad (18)$$

work, where the support sizes are measured after the recursive child updates and before the pull at u . The bound is $O(n)$ in the worst case.

Proof. A standard interval decomposition visits $O(\log n)$ fully covered canonical nodes and has at most two partially covered nodes at each depth. Full-cover work is constant per canonical node. Every partial node performs one weighted persistent union, whose cost is $O(\mathcal{J}(d_{u_L}, d_{u_R}))$ by Lemma 23.

For the worst-case bound, $\mathcal{J}(a, b) = O(a + b)$ and $a + b \leq |I_u|$. At depth j , at most two partial nodes occur and

each has interval length $O(n/2^j)$. Therefore

$$\sum_{u \in \partial I} |I_u| = O\left(\sum_{j \geq 0} \frac{n}{2^j}\right) = O(n).$$

Substitution into (18) proves the claim. $\square$

The support-sensitive expression can be much smaller than n . For example, if every partial pull merges maps with at most s distinct keys on the smaller side, the update cost is

$$O\left(\log n + s \log n \log\left(\frac{n}{s} + 1\right)\right).$$

Long index intervals containing many repeated values can therefore be updated without scanning their elements, even though adversarial all-distinct states retain a linear fallback scale.

10.4 Exact range queries by temporary meld

Let a query interval $Q = [l, r]$ have the canonical index-tree decomposition

$$\mathcal{C}(Q) = \{v_1, \dots, v_k\}, \quad k = O(\log n).$$

The maps T_{v_i} already contain current values, including all lazy shifts at their roots. Construct a temporary histogram for Q by a balanced binary schedule of Union_+ operations and return the $\mathcal{F}$ field of the final root. The stored maps are not changed.

Define

$$D_Q = \sum_{i=1}^k d_{v_i}.$$

Since the canonical index intervals are disjoint, $D_Q \leq M$, where $M = r - l + 1$.

► **Theorem 27 (Support-sensitive exact range query).** *A range query can be answered exactly in*

$$O(\log n + k + D_Q \lceil \log_2 k \rceil) \quad (19)$$

work and the same order of temporary persistent-map space. For fixed-width integer values, combining this route with radix-statistical extraction gives

$$O(\log n + \min\{M, k + D_Q \lceil \log_2 k \rceil\}) \quad (20)$$

query time.

Proof. Collecting the canonical nodes costs $O(\log n + k)$. Arrange their roots as leaves of a balanced merge tree. At one merge level, the input maps are disjoint as objects, and the sum of their support sizes is at most D_Q ; because $\mathcal{J}(a, b) = O(a + b)$, the total union work on that level is $O(D_Q)$. There are $\lceil \log_2 k \rceil$ levels. The root summary gives the exact pairwise sum. The radix route independently costs $O(M + \log n)$, so the planner may choose the smaller bound. $\square$

When Q itself is one stored segment-tree node, no histogram merge is needed: its answer is the stored field F_u . More generally, the shift-meld route is advantageous when the canonical nodes have small distinct-value support, while the radix route protects the worst case.

10.5 Construction, space, and relation to buffering

Building the index tree bottom-up by weighted unions takes $O(n \log n)$ work in the all-distinct worst case. Each input element contributes to the histogram of every index-tree ancestor, so the live representation has $O(n \log n)$ logical map entries and uses $O(n \log n)$ space. Path-copy nodes created by an operation are bounded by that operation's meld work; after obsolete roots are reclaimed, they do not accumulate across operations. The three-dimensional Manhattan structure uses three independent copies.

► Theorem 28 (Concrete exact shift-meld structure).

For the one-dimensional dynamic pairwise L_1 problem, the shift-meld histogram segment tree has $O(n \log n)$ pre-processing work and live space, supports range additions by (18), and supports exact range queries by (19). With the fixed-width radix fallback, the query bound is (20). All answers are exact.

The structure corresponds to immediate materialization of every partially changed ancestor. In the component-log language of Section 9, it is the threshold-one endpoint: after each partial change, the parent histogram is compacted at once. It also supplies a concrete guaranteed compaction route for larger buffers. A stale node can always discard its symbolic log and rebuild its materialized map by melding its two current child maps in $O(\mathcal{J}(d_{u_L}, d_{u_R}))$ work. Matrix-CDQ may reduce the cost of evaluating or batching pending interactions before such a flush, but correctness and termination no longer depend on an unspecified rebuild primitive.

The remaining challenge is therefore quantitative rather than representational. The immediate shift-meld route can spend linear work near the root, while a deferred component route must control log growth and query-triggered flushes. A stronger dynamic result would combine the guaranteed materialized baseline here with the sparse hierarchical interaction factors of Section 8 so that repeated large-support pulls are answered from reusable factors rather than reconstructed from scratch.

11 Index-CDQ Pair-Space Oracle

Value-CDQ attacks value fragmentation. A complementary idea is to decompose pair-space by index using a

CDQ or segment-tree skeleton.

Build a balanced binary tree over indices. For every pair $i < j$, assign the pair to the lowest node u such that i lies in the left child of u and j lies in the right child of u . This is the LCA of the two leaves. At node u , store the pair point

$$(i, j, a_j - a_i).$$

► **Lemma 24 (LCA pair assignment).** *Each pair (i, j) , $i < j$, is stored in exactly one index-CDQ node. Hence the total number of stored pair points over all nodes is $O(n^2)$.*

Proof. The two leaves for i and j have a unique lowest common ancestor. At this node they lie in different children, and at every strict descendant they do not both appear. Thus the pair is assigned once. Since the total number of index pairs is $\binom{n}{2}$, the total number of stored pair points is $O(n^2)$. $\square$

In the static case, a query interval can be evaluated recursively. At each visited node, the only cross contribution is between the part of the query in the left child and the part in the right child. This is answered by the node-local pair-space oracle.

In the dynamic lazy model, clean nodes carry scalar tags. Mixed or dirty nodes are expanded recursively. Let χ_Q be the number of clean node-pair Cross calls made by this recursive evaluation.

► **Theorem 29 (Index-CDQ exact evaluation).** *Under lazy range translations, an index-CDQ exact query can be answered in*

$$O(\chi_Q P(n) + D_Q R'(n) + \text{Ord}(D_Q))$$

time, where $P(n)$ is the node-local Cross query cost, D_Q is the number of dirty elements in the query, and $R'(n)$ is the point-to-range correction cost.

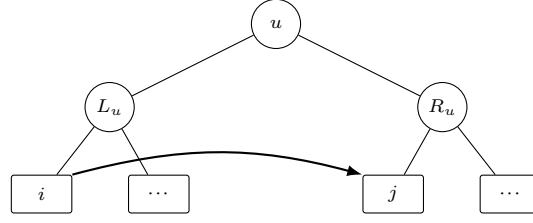
The parameter χ_Q measures hierarchical Cross expansion and may still be quadratic in the number of fragmented runs. The structural traversal needed to expose those runs, however, admits a non-tautological bound. Let b_Q be the number of adjacent positions inside the query at which the accumulated clean offset changes, and set $\beta_Q = \max\{1, b_Q\}$.

► **Lemma 25 (Index-boundary entropy).** *The number of mixed segment-tree nodes whose queried portion contains more than one clean offset is*

$$O\left(\log n + \beta_Q \left(1 + \log \frac{n}{\beta_Q}\right)\right).$$

Consequently, the maximal uniform frontier of the query can be extracted within the same bound, excluding the cost of evaluating Cross interactions between frontier nodes.

Proof. At depth h , mixed nodes are disjoint and each contains at least one of the b_Q offset boundaries, so there are at most $\min\{2^h, b_Q\}$ such nodes. Summing over



the pair (i, j) is stored at the first split node $u = \text{lca}(i, j)$

Figure 9: LCA assignment in the index-CDQ pair-space oracle. Each pair is stored once, at the node whose children separate its two indices.

depths gives

$$\sum_{h=0}^{\lceil \log n \rceil} \min\{2^h, \beta_Q\} = O\left(\beta_Q \left(1 + \log \frac{n}{\beta_Q}\right)\right).$$

The two index endpoints contribute the additional $O(\log n)$ canonical boundary paths. Stopping whenever a node is uniform produces the maximal frontier. $\square$

This bound controls the index skeleton rather than χ_Q itself: it proves that low alternation yields a small set of atoms, but a separate argument is still required to bound the number of hard interactions among those atoms.

12 Adaptive Exact Query Planner

The exact oracle should not use a single evaluation path. It should select the cheapest representation of the same exact quantity.

Let

$$M = r - l + 1.$$

Write $\text{Ord}(M)$ for the cost of direct exact order recovery: $O(M \log M)$ for arbitrary comparable values and $O(M)$ for fixed-width integers by Section 4.2. Let g be the number of clean shifted runs, let D_Q be the dirty mass, let η_Q be the full value-factor active-incidence complexity, let $\zeta_Q \leq \eta_Q$ be the interaction-only factor sparsity when run-internal sums are stored, let χ_Q be the index-CDQ cross-expansion complexity, and let ω_Q be the number of hard overlap interactions after separation pruning. Then the clean exact query cost is modeled as

$$Q_{\text{clean}}(l, r) = O\left(\min\{\text{Ord}(M), g^2 P(n), \zeta_Q R(n), \chi_Q P(n), \omega_Q P(n)\}\right).$$

After dirty correction, the full one-dimensional exact cost is

$$Q(l, r) = Q_{\text{clean}}(l, r) + O(D_Q R'(n) + \text{Ord}(D_Q)).$$

This is the exact part of the Magnus Model: exact difficulty is separated into offset fragmentation, value-support entropy, hierarchical factor sparsity, index fragmentation, value-overlap residue, and dirty perturbation. The oracle chooses the cheaper side of the index-value fragmentation duality, after pruning away interactions whose value geometry is already easy.

12.1 Rebuilding Policy

The model does not avoid rebuilding entirely. It avoids blind global rebuilding. The natural policy is:

- full-cover updates modify only lazy tags;
- partial updates create dirty elements or dirty sub-objects;
- if a block or CDQ node becomes too dirty, rebuild that local object;
- if dirty mass becomes global, rebuild the global base structure.

Thus the exact model is adaptive. In adversarial cases it falls back to sorting or rebuilding, but in low-fragmentation cases it can be much faster than the plain epoch oracle.

13 Ordering Barrier and a 3SUM-Based Exact Frontier

The exact approaches above either require near-linear query time, large pair-space preprocessing, or an $O(T^2)$ enumeration of offset-segment pairs. We first explain the obstacle structurally and then give a direct fine-grained conditional barrier.

For a multiset S with

$$C(t) = \#\{v \in S : v \leq t\},$$

one has

$$\sum_{i < j} |v_i - v_j| = \int_{-\infty}^{\infty} C(t)(|S| - C(t)) dt.$$

Thus exact computation depends on the CDF breakpoints of the query multiset. Range additions shift parts of the value distribution, and different shifted parts may interleave in complicated ways. This is the dynamic ordering barrier.

Another view uses the cross function

$$C_{A,B}(\lambda) = \sum_{x \in A, y \in B} |y - x + \lambda|.$$

It is convex and piecewise linear, with breakpoints at $\lambda = x - y$. In the worst case there are $|A||B|$ distinct breakpoints. Pair-space preprocessing stores these events explicitly; adaptive exact methods attempt to avoid paying for all of them unless the workload exposes them.

Regime	Structural feature	Method	Cost	
Small interval	M small	direct order recovery	$O(\text{Ord}(M))$	
Low offset fragmentation	g small	run-pair Cross	$O(g^2 P(n))$	
Low factor sparsity	$\zeta_Q \ll g^2$	exact hierarchical factorization	$O(\zeta_Q R(n))$	
Hierarchical fragmentation	$\chi_Q \ll g^2$	index-CDQ Cross	$O(\chi_Q P(n))$	
Separable values	$\omega_Q \ll g^2$	sieve-pruned closed forms	$O(\omega_Q P(n))$	
Low dirty mass	D_Q small	multiset-regrouped correction	$O(D_Q R'(n))$	+
Adversarial fall-back	all parameters large	radix/sort values	$O(\text{Ord}(M))$	

Table 2: Fragmentation-aware exact query paths.

13.1 Tripartite 3SUM Hypothesis

We use the integer tripartite formulation of 3SUM. The input consists of three sets

$$A, B, C \subseteq [-W, W] \cap \mathbb{Z}, \quad |A| = |B| = |C| = n,$$

where $W = n^{O(1)}$, and the task is to decide whether there exist

$$a \in A, \quad b \in B, \quad c \in C$$

such that $a + b + c = 0$. The *Tripartite 3SUM Hypothesis* states that, for every constant $\varepsilon > 0$, no algorithm solves all such instances in $O(n^{2-\varepsilon})$ time on a word-RAM with $\Theta(\log n)$ -bit words. This is a standard fine-grained formulation of the 3SUM conjecture [17, 18, 19].

13.2 Discrete curvature of the cross function

The reduction uses the fact that the absolute-value cusp becomes a point indicator after a discrete second difference. Section 9 used the same identity algorithmically as the curvature-correlation transform; here it is used adversarially to recover exact pair-sum multiplicities.

► **Lemma 26 (Discrete second difference of absolute value).** For integers s, t ,

$$|s - (t+1)| - 2|s - t| + |s - (t-1)| = \begin{cases} 2, & s = t, \\ 0, & s \neq t. \end{cases}$$

Proof. If $s = t$, the left-hand side is $1 - 0 + 1 = 2$. If $s \leq t - 1$ or $s \geq t + 1$, all three arguments lie on one affine branch of the absolute-value function, whose second difference is zero. $\square$

For two integer sets A, B , define the parametric cross function

$$H(t) = \sum_{a \in A, b \in B} |a + b - t|.$$

Summing the lemma over all pairs gives the exact identity

$$\frac{H(t+1) - 2H(t) + H(t-1)}{2} = \#\{(a, b) \in A \times B : a + b = t\}. \quad (21)$$

Thus three neighboring evaluations of H recover the multiplicity of any prescribed pair sum.

13.3 Reduction to dynamic pairwise L_1 aggregation

► **Theorem 30 (Tripartite-3SUM reduction).** Consider an exact data structure for the one-dimensional dynamic pairwise L_1 problem on positive integer arrays of length N . Let its preprocessing time be $P(N)$, its amortized update time be $U(N)$, and its amortized query time be $Q(N)$. Then Tripartite 3SUM on three sets of size n can be solved in

$$P(2n) + O(n(U(2n) + Q(2n)) + n \log n)$$

time.

The reduction uses only the following restricted operation sequence:

- every update adds an integer to the fixed suffix $[n+1, 2n]$;
- every query asks for the whole range $[1, 2n]$;
- every array entry remains a positive integer.

Proof. Let $A, B, C \subseteq [-W, W] \cap \mathbb{Z}$ be a Tripartite 3SUM instance and choose

$$M = 3W + 3.$$

For a translation parameter $t \in [-W-1, W+1]$, form the length- $2n$ array

$$X(t) = (M + a_1, \dots, M + a_n, M - b_1 + t, \dots, M - b_n + t).$$

Every entry is positive: $M + a_i \geq 2W + 3$, and $M - b_j + t \geq W + 2$. Moving from parameter t to t' requires exactly one range addition of $t' - t$ to the fixed suffix $[n+1, 2n]$.

Let $F(t)$ be the whole-array pairwise L_1 query on $X(t)$. Pairs internal to the first block contribute

$$I_A = \sum_{1 \leq i < j \leq n} |a_i - a_j|,$$

and pairs internal to the second block contribute

$$I_B = \sum_{1 \leq i < j \leq n} |b_i - b_j|.$$

Both quantities are independent of t . The cross-block contribution is

$$\sum_{a \in A, b \in B} |(M + a) - (M - b + t)| = \sum_{a \in A, b \in B} |a + b - t| = H(t).$$

Hence

$$H(t) = F(t) - I_A - I_B.$$

The constants I_A, I_B can be computed in $O(n \log n)$ time by sorting and prefix sums.

For each $c \in C$, evaluate $H(-c-1)$, $H(-c)$, and $H(-c+1)$. Equation (21) determines whether there is a pair $(a, b) \in A \times B$ with $a+b=-c$. Therefore the original instance is positive exactly when one of these second differences is nonzero. Across all $c \in C$, the algorithm performs $O(n)$ suffix updates and $O(n)$ whole-array queries after one preprocessing phase, giving the stated running time. $\square$

► **Corollary 5 (Conditional preprocessing–operation tradeoff).** *Under the Tripartite 3SUM Hypothesis, there do not exist constants $\varepsilon_P, \varepsilon_O > 0$ for which an exact structure simultaneously satisfies*

$$\begin{aligned} P(N) &= O(N^{2-\varepsilon_P}), \\ U(N) + Q(N) &= O(N^{1-\varepsilon_O}). \end{aligned}$$

This remains true for positive integer arrays, fixed-suffix updates, and whole-array queries.

Proof. Substituting the two assumed bounds into the reduction yields a Tripartite 3SUM algorithm running in

$$O(n^{2-\varepsilon_P} + n^{2-\varepsilon_O} + n \log n) = O(n^{2-\varepsilon})$$

for $\varepsilon = \min\{\varepsilon_P, \varepsilon_O\} > 0$, contradicting the hypothesis. $\square$

Interpretation. The reduction isolates the hard object more sharply than the structural breakpoint argument alone. Arbitrary intervals, many fragmented lazy pieces, and multidimensional geometry are not required for conditional hardness. Two fixed blocks with one scalar relative translation already encode exact pair-sum counting through the curvature of H . The result is conditional rather than an unconditional cell-probe lower bound, and it leaves room for faster algorithms on structured inputs or with near-quadratic preprocessing. It nevertheless explains why a general exact structure with both light preprocessing and strongly sublinear operations would constitute a fine-grained breakthrough.

13.4 Conditional optimality of linear exact order recovery

The upper and lower bounds now meet at the correct scale for whole-array word-integer queries. Section 4.2 gives $P(N) = O(N)$, $U(N) = O(\log N)$, and $Q(N) = O(N)$. The conditional tradeoff implies that, under the Tripartite 3SUM Hypothesis and truly subquadratic preprocessing, $U(N) + Q(N) = N^{1-o(1)}$. In particular, when updates are polylogarithmic, exact whole-array queries require $N^{1-o(1)}$ time in the conditional sense. Thus the radix-statistical query is optimal up to subpolynomial factors for this restricted setting.

This conclusion must be interpreted carefully. The lower bound does not prove that every exact algorithm literally sorts, nor does it rule out improvements on structured instances. It says that any general exact method

with light preprocessing must still pay near-linear cost somewhere in the update–query pair. Fragmentation, separation, and memoization can avoid that cost on compressible workloads; near-quadratic pair-space compilation can move the cost into preprocessing; approximation can remove it by no longer recovering the cusp exactly.

The Magnus Model therefore has three complementary routes. Its direct exact side recovers order in linear word-RAM time. Its adaptive exact side exploits structure or compilation to beat that fallback on favorable workloads. Its approximate side replaces the absolute-value cusp by a finite spectral expansion, turning range additions into phase multiplications and permitting polylogarithmic approximate update and query time.

14 Algebraic CDQ Approximation Model

The exact model pays for dynamic ordering. The approximate model replaces exact ordering by finite-dimensional algebra. The common template is a feature map $\Phi: \mathbb{R} \rightarrow \mathbb{R}^d$ and a matrix A such that

$$|x - y| \approx \Phi(x)^\top A \Phi(y),$$

and a translation representation

$$\Phi(x + \Delta) = T_\Delta \Phi(x).$$

For a segment-tree or CDQ node u , store

$$S_u = \sum_{i \in u} \Phi(x_i), \quad \text{Ans}_u \approx \sum_{i < j, i, j \in u} |x_i - x_j|.$$

If u has children L, R , then

$$S_u = S_L + S_R,$$

and

$$\text{Ans}_u = \text{Ans}_L + \text{Ans}_R + S_L^\top A S_R.$$

A full-cover range translation does not change the internal pairwise distances, so Ans_u is unchanged. Only the external feature sum changes:

$$S_u \leftarrow T_\Delta S_u.$$

► **Theorem 31 (Algebraic CDQ approximation).** *Assume a feature representation of dimension d with translation cost $C_T(d)$ for applying T_Δ , and bilinear cross cost $C_A(d)$ for evaluating $S_L^\top A S_R$. Then a lazy segment tree or CDQ segment structure maintains the approximate one-dimensional range pairwise L_1 aggregate with*

$$O(dn) \quad \text{space,}$$

$$O(C_T(d) \log n) \quad \text{range-update time,}$$

and

$$O(C_A(d) \log n) \quad \text{range-query time.}$$

Proof. A segment-tree range operation visits $O(\log n)$ canonical nodes. A full-cover update applies one translation operator to each visited canonical node. A query combines the stored feature sums and internal answers of $O(\log n)$ nodes in index order; each cross term is a bilinear feature interaction. The storage bound follows from storing d -dimensional feature sums at $O(n)$ nodes. $\square$

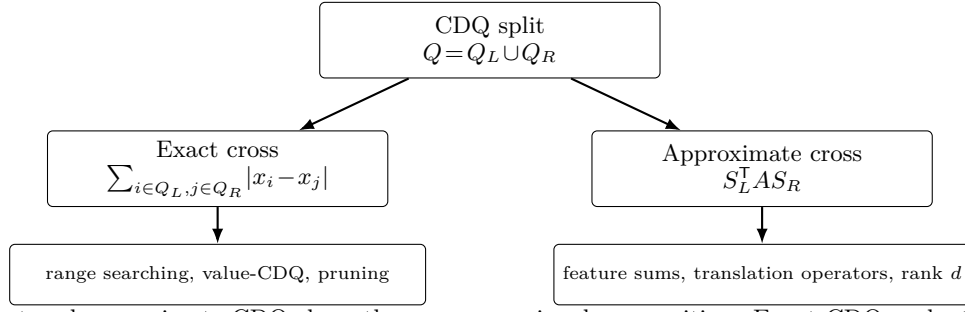


Figure 10: Exact and approximate CDQ share the same recursive decomposition. Exact CDQ evaluates cross terms by range-searching, value decomposition, and pruning; approximate CDQ replaces cross terms by bilinear interactions between finite-dimensional feature sums.

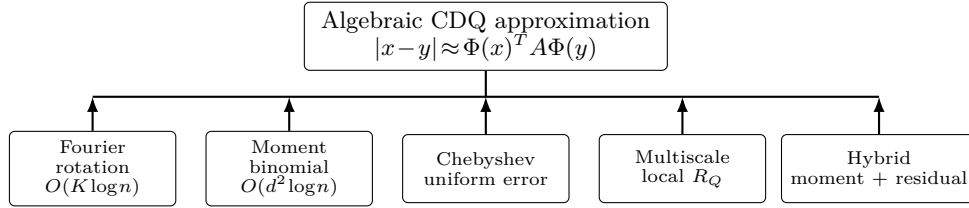


Figure 11: Approximation families under the algebraic CDQ template. They differ by translation operator, bilinear cost, and error model.

This theorem classifies approximate methods by algebraic rank rather than by fragmentation. Important special cases are:

- **Fourier-CDQ.** Features are $(\cos tx, \sin tx)$. Translation is a rotation, and both update and cross evaluation are linear in the number of frequencies.
- **Moment-CDQ.** Features are $(1, x, x^2, \dots, x^d)$. Translation is the binomial transform, giving $O(d^2 \log n)$ update/query time.
- **Chebyshev-CDQ.** Chebyshev bases improve numerical stability and give clean uniform approximation statements on a chosen scale.
- **Multiscale-CDQ.** Several scales are maintained, and a query chooses the smallest scale covering its observed value range. This ties the additive error to the local range R_Q rather than a global range.
- **Hybrid-CDQ.** A low-degree moment or Chebyshev approximation handles the global shape, while spectral features approximate the residual.

Thus exact CDQ classifies fragmentation, while approximate CDQ classifies algebraic rank.

14.1 Fourier/Spectral Specialization

The spectral Magnus Model is the fastest specialization. It starts from the identity

$$|u| = \frac{2}{\pi} \int_0^\infty \frac{1 - \cos(tu)}{t^2} dt.$$

Instead of evaluating the integral exactly, choose D frequencies and weights

$$(t_1, w_1), \dots, (t_D, w_D),$$

and define the finite kernel

$$\phi_D(u) = \sum_{s=1}^D w_s (1 - \cos(t_s u)).$$

► **Definition 4 (Uniform kernel approximation).** We say that ϕ_D is an ε -approximation to $|u|$ on $[-M, M]$ if

$$\max_{|u| \leq M} |\phi_D(u) - |u|| \leq \varepsilon.$$

The frequencies and weights may be obtained by numerical quadrature, cosine approximation, Chebyshev approximation, Remez-type approximation, or other one-dimensional approximation methods. The data structure only requires that such a kernel is given.

14.2 An Explicit Fourier Kernel

For completeness, we record one concrete deterministic kernel. On the interval $[-M, M]$, the cosine series of $|u|$ is

$$|u| = \frac{M}{2} - \frac{4M}{\pi^2} \sum_{\substack{k \geq 1 \\ k \text{ odd}}} \frac{\cos(k\pi u/M)}{k^2}.$$

Since

$$\sum_{\substack{k \geq 1 \\ k \text{ odd}}} \frac{4M}{\pi^2 k^2} = \frac{M}{2},$$

this can be written in the Magnus Model form by taking

$$k_s = 2s - 1, \quad t_s = \frac{k_s \pi}{M}, \quad w_s = \frac{4M}{\pi^2 k_s^2},$$

and

$$\phi_D(u) = \sum_{s=1}^D w_s (1 - \cos(t_s u)).$$

► **Lemma 27 (Odd-cosine representation).** On $[-M, M]$, the function $|u|$ admits the cosine representation

$$|u| = \sum_{s=1}^D \frac{4M}{\pi^2 (2s-1)^2} \left(1 - \cos\left(\frac{(2s-1)\pi u}{M}\right) \right).$$

Proof. This is the standard cosine series for the even function $|u|$ on $[-M, M]$, rewritten using only odd harmonics. The constant term equals the sum of the odd-harmonic weights, so the series can be expressed as a nonnegative combination of terms $1 - \cos(t_s u)$. $\square$

► **Lemma 28 (Fourier truncation bound).** For $u \in [-M, M]$, the truncated odd-harmonic kernel above satisfies the conservative bound

$$|\phi_D(u) - |u|| \leq \frac{4M}{\pi^2(2D-1)}.$$

Proof. The omitted tail is

$$\sum_{s>D} w_s (1 - \cos(t_s u)).$$

Since $0 \leq 1 - \cos(t_s u) \leq 2$, its absolute contribution is at most

$$2 \sum_{s>D} \frac{4M}{\pi^2(2s-1)^2} \leq \frac{8M}{\pi^2} \int_D^\infty \frac{dx}{(2x-1)^2} = \frac{4M}{\pi^2(2D-1)}.$$

$\square$

This explicit kernel is not claimed to be optimal. Its purpose is to make the approximation theorem directly executable and to give a simple $O(M/D)$ error scale for implementations.

15 Spectral Pairwise Formula

For an interval $[l, r]$, let $m = r - l + 1$, and define

$$Z_s(l, r) = \sum_{i=l}^r e^{it_s a_i}.$$

The key point is that one complex sum stores all pairwise cosine interactions at the same frequency. We split this into a norm expansion and then the pairwise identity used by the data structure.

► **Lemma 29 (Frequency norm expansion).** For each frequency t_s ,

$$|Z_s(l, r)|^2 = m + 2 \sum_{l \leq i < j \leq r} \cos(t_s(a_i - a_j)).$$

Proof. Expanding the squared modulus gives

$$|Z_s|^2 = Z_s \overline{Z_s} = \sum_{i=l}^r \sum_{j=l}^r e^{it_s(a_i - a_j)}.$$

The diagonal terms with $i = j$ contribute m . For every unordered pair $i < j$, the two off-diagonal terms are complex conjugates,

$$e^{it_s(a_i - a_j)} + e^{it_s(a_j - a_i)} = 2\cos(t_s(a_i - a_j)).$$

Adding the diagonal and off-diagonal contributions proves the identity. $\square$

► **Lemma 30 (Spectral pairwise identity).** For each frequency t_s ,

$$m^2 - |Z_s(l, r)|^2 = 2 \sum_{l \leq i < j \leq r} (1 - \cos(t_s(a_i - a_j))).$$

Proof. By the frequency norm expansion,

$$|Z_s|^2 = m + 2 \sum_{l \leq i < j \leq r} \cos(t_s(a_i - a_j)).$$

Since $m^2 = m + 2\binom{m}{2}$, subtracting the displayed identity from m^2 leaves exactly two copies of $1 - \cos(t_s(a_i - a_j))$ for every unordered pair. $\square$

Therefore, the approximate one-dimensional answer is

$$\widehat{F}(l, r) = \frac{1}{2} \sum_{s=1}^D w_s (m^2 - |Z_s(l, r)|^2).$$

► **Theorem 32 (Additive error bound).** Assume that all differences $a_i - a_j$ occurring in a query interval lie in $[-M, M]$, and that

$$|\phi_D(u) - |u|| \leq \varepsilon$$

for all $u \in [-M, M]$. Then

$$|\widehat{F}(l, r) - F(l, r)| \leq \varepsilon \binom{m}{2}.$$

Proof. For every pair (i, j) ,

$$|\phi_D(a_i - a_j) - |a_i - a_j|| \leq \varepsilon.$$

Summing over all $\binom{m}{2}$ pairs gives the result. $\square$

16 Lazy Spectral Segment Tree

We now describe the data structure for the one-dimensional approximate problem.

For each frequency t_s , define

$$z_{i,s} = e^{it_s a_i}.$$

A segment tree node u stores

$$Z_{u,s} = \sum_{i \in u} z_{i,s}$$

for all $s = 1, \dots, D$.

16.1 Range Addition

Suppose we add Δ to all a_i in a segment. Then

$$e^{it_s(a_i + \Delta)} = e^{it_s \Delta} e^{it_s a_i}.$$

Therefore, for a segment tree node fully covered by the update,

$$Z_{u,s} \leftarrow e^{it_s \Delta} Z_{u,s}.$$

The lazy tag of the node is also multiplied by

$$e^{it_s \Delta}.$$

Thus range addition becomes ordinary lazy propagation with complex multiplicative tags.

16.2 Range Query

To answer Query(l, r), we query the segment tree and obtain

$$Z_s(l, r) = \sum_{i=l}^r e^{it_s a_i}$$

for all s . Then compute

$$\hat{F}(l, r) = \frac{1}{2} \sum_{s=1}^D w_s (m^2 - |Z_s(l, r)|^2).$$

► **Theorem 33 (One-dimensional spectral complexity).** *The Lazy Spectral Segment Tree supports each range addition and each approximate range pairwise L_1 query in*

$$O(D \log n)$$

time and uses

$$O(Dn)$$

space.

Proof. Each segment tree operation touches $O(\log n)$ nodes. At each touched node, all D frequencies are updated or accumulated. Thus the time complexity is $O(D \log n)$. Each of $O(n)$ segment tree nodes stores D complex values, giving $O(Dn)$ space. $\square$

17 Three-Dimensional Approximate Manhattan Sum

For the original three-dimensional problem, build three independent Lazy Spectral Segment Trees, one for each coordinate.

For a range translation

$$(\Delta_x, \Delta_y, \Delta_z),$$

apply the one-dimensional spectral update with Δ_x , Δ_y , and Δ_z separately.

For a query interval $[l, r]$, compute

$$\hat{F}_x(l, r), \quad \hat{F}_y(l, r), \quad \hat{F}_z(l, r),$$

and return

$$\widehat{\text{Ans}}(l, r) = \hat{F}_x(l, r) + \hat{F}_y(l, r) + \hat{F}_z(l, r).$$

► **Theorem 34 (Three-dimensional spectral complexity and error).** *Assume that each one-dimensional kernel approximation has uniform error at most ε on the relevant difference range. Then the three-dimensional Lazy Spectral Segment Tree supports each operation in*

$$O(3D \log n)$$

time and uses

$$O(3Dn)$$

space. For a query of length m , the additive error is at most

$$3\varepsilon \binom{m}{2}.$$

Proof. The complexity follows from maintaining three independent one-dimensional structures. The error bound follows by summing the one-dimensional additive error bounds over the three coordinates. $\square$

18 Discussion on Kernel Choice

The data structure is independent of the specific method used to choose the frequencies and weights. It only assumes a finite approximation

$$\phi_D(u) = \sum_{s=1}^D w_s (1 - \cos(t_s u))$$

to $|u|$ on a range $[-M, M]$.

Several strategies are possible.

■ **Quadrature.** Start from

$$|u| = \frac{2}{\pi} \int_0^\infty \frac{1 - \cos(tu)}{t^2} dt$$

and apply numerical quadrature after truncating the integral.

- **Cosine approximation.** After scaling the difference range to a bounded interval, approximate $|u|$ by a finite cosine expansion.
- **Chebyshev or Remez approximation.** Approximate $|u|$ using standard one-dimensional approximation methods, then convert the approximation into a usable finite feature representation.
- **Random or deterministic spectral features.** Use sampled frequencies and weights to obtain a sketching-style approximation.

In this note, the algorithmic guarantees are conditional on the uniform approximation error of ϕ_D .

19 Exact Far Field and Approximate Near-Field Residues

The absolute-distance kernel needs no approximation on ordered separated blocks. If two shifted value intervals satisfy $\max I_A \leq \min I_B$, then $K_{A,B}(x, y) = y - x$ and the full block interaction is the exact rank-two expression

$$|A| \sum_{y \in B} y - |B| \sum_{x \in A} x.$$

Therefore the far field should be evaluated exactly; spectral error is spent only on unresolved blocks that still straddle or approach the cusp at $x = y$.

Suppose a hierarchical router partitions the unordered query pairs into exact separated blocks and approximate residue blocks $\mathcal{A}_1, \dots, \mathcal{A}_s$. Let p_r be the number of pairs in $\mathcal{A}_r$, and suppose all their differences lie in $[-M_r, M_r]$. The block may use any translation-closed kernel ϕ_r with uniform error ε_r on that interval.

► **Theorem 35 (Localized hybrid error).** *If every separated block is evaluated exactly and residue block $\mathcal{A}_r$ is evaluated by ϕ_r , then*

$$|\tilde{F} - F| \leq \sum_{r=1}^s p_r \varepsilon_r.$$

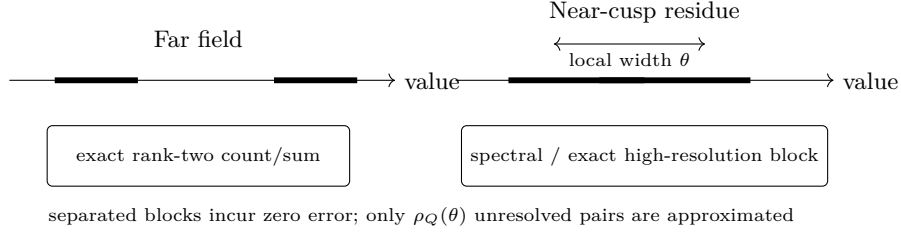


Figure 12: Exact-far/approximate-near routing. Ordered separated blocks are exact rank two; spectral approximation is localized to the near-cusp residue.

For the explicit D -term odd-cosine kernel, if $M_r \leq \theta$ for all residue blocks and $\rho_Q(\theta) = \sum_{r=1}^s p_r$, then

$$|\tilde{F} - F| \leq \frac{4\theta}{\pi^2(2D-1)} \rho_Q(\theta).$$

Proof. Exact blocks contribute no error. On residue block r , each of its p_r pair contributions changes by at most ε_r ; the triangle inequality gives the first bound. The second follows by substituting the Fourier truncation bound with interval radius $M_r \leq \theta$. $\square$

A schematic hybrid cost is

$$Q_{\text{hybrid}} = T_{\text{route}} + N_{\text{far}} C_{\text{sum}} + \sum_{r=1}^s C_{\text{alg}}(D_r),$$

where exact separated blocks cost only count/sum contractions, while $C_{\text{alg}}(D_r) = O(D_r)$ for Fourier features. The guarantee improves simultaneously in two ways: the approximation interval is local rather than global, and the pair count is ρ_Q rather than $\binom{m}{2}$.

20 Integrated Interaction-Routing Model

The final model combines exact, approximate, pruning, and compiled-reuse paths. Define the regrouped dirty cost

$$T_D(Q) = O(D_Q R'(n) + \text{Ord}(D_Q)).$$

Then the exact route is modeled as

$$Q_{\text{exact}}(l, r) = O\left(\min\{\text{Ord}(M), g^2 P(n), \zeta_Q R(n), \chi_Q P(n), \omega_Q P(n)\} + T_D(Q)\right).$$

The direct radix evaluator may sort and scan or use the divide-and-aggregate recursion; both are linear for fixed-width words. Cold hard interactions are served by the selected exact evaluator until the rent-or-buy threshold justifies compilation, after which kinetic fingers exploit locality in the relative lazy offset.

For an approximate answer, separated blocks remain on the exact count/sum route. If residue blocks r contain p_r pairs, use local feature dimensions D_r , and have difference radii M_r , then

$$Q_{\text{hybrid}} = O\left(T_{\text{route}} + N_{\text{far}} C_{\text{sum}} + \sum_r C_{\text{alg}}(D_r) + T_D(Q)\right),$$

$$|\tilde{F} - F| \leq \sum_r p_r \varepsilon_r.$$

Thus approximation is no longer a generic far-field route: it is an explicitly budgeted near-cusp residue after all exact rank-two blocks have been removed.

In short, pruning removes exact rank-two blocks, CDQ decomposes the unresolved exact kernel, dirty regrouping repairs small update residues without a quadratic term, rent-or-buy decides when a hard interaction deserves compilation, kinetic fingers reuse its breakpoint order, and spectral features approximate only the remaining near-cusp blocks.

21 Implementation Remarks

Exact integer path. For the radix-statistical fallback, store values in a flat contiguous buffer and flip the sign bit before processing signed integers. One implementation performs stable counting passes and a final rank scan; the direct alternative performs an MSD divide-and-aggregate recursion, accumulating cross-bucket count/sum terms without emitting sorted output. The pairwise sum should be accumulated in a wider type than the input word: even when each value fits in 64 bits, $\binom{m}{2} \max_{i,j} |a_i - a_j|$ may require more than 64 bits. Lazy extraction should traverse the query subtree once rather than issuing m independent point queries.

Shift-meld histogram path. The weighted ordered map should be immutable or copy-on-write, because a parent histogram and its child histograms may share search-tree nodes. A map node stores key multiplicity, distinct support, total multiplicity, weighted value sum, internal pairwise distance, balance metadata, and a lazy key-shift tag. Equal-key multiplicities must be combined during union; zero-count nodes are removed. Obsolete roots should be reclaimed after each operation, otherwise path copying turns the live-space bound into a historical-space bound. A query uses temporary merge roots and discards them immediately after reading the final pairwise field.

Compiled interaction path. A compiled Cross object stores exact object/version identifiers, sorted breakpoints, prefix sums, the previous threshold rank, and a rental counter. Finger search must fall back to ordinary binary search when the displacement estimate is poor. Compilation is triggered only after the rent-or-buy threshold; measured costs may be used in engineering implementations, while a certified analysis uses upper

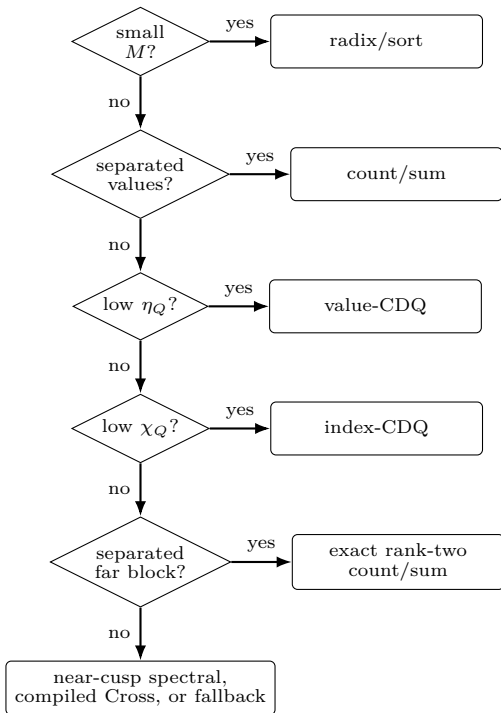


Figure 13: A schematic decision tree for the interaction router. The actual implementation may evaluate estimated costs rather than following a fixed order.

Component	Parameter	Evaluator	Role
Raw interval	M	radix/statistical or comparison order recovery	exact fallback
Offset fragmentation	g	run-pair Cross	exact low-run path
Value fragmentation	η_Q	shifted-run value-CDQ	exact value path
Index hierarchy	χ_Q	index-CDQ	exact recursive path
Overlap residue	ω_Q	sieve-pruned Cross	exact hard core
Dirty perturbation	D_Q	correction/local rebuild	exact update residue
Algebraic rank	d, K	Fourier/moment/Chebyshev CDQ	approximate path
Near-cusp residue	ρ_Q	local spectral or exact high-resolution routine	only approximate error source
Compiled state	versions, cursor, rental count	rent-or-buy kinetic Cross	amortization and locality

Table 3: The integrated Magnus Model routes each interaction by its structural state.

bounds B_e, C_e, q_e . Cursor state is discarded when either object version changes.

Dirty residue path. The clean baseline should exclude dirty elements from the clean histogram and retain their proxy multiset separately. Compute the correction through $F(V_D) - F(O_D)$ and two dirty-clean Cross terms. Do not apply independent point corrections followed by a dirty-dirty repair, because that reintroduces the avoidable quadratic term.

Lazy-incremental exact path. An implementation of the transform in Section 9 should distinguish logical shifts from materialized value movement. Full-cover nodes carry only scalar tags. Partial updates append signed references to shared subhistogram objects rather than copying supports. Correlation components require a strict memory budget and deterministic compaction threshold; otherwise a favorable update sequence can still create an unbounded symbolic log. The exact answer must be accumulated in a sufficiently wide signed integer type, while Laurent exponents and relative tags require explicit overflow checks.

Spectral path. The implementation should avoid using `std::complex` in performance-sensitive settings. A custom complex type with two floating-point fields is usually faster:

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i.$$

For each range update and each frequency, compute

$$\cos(t_s \Delta), \quad \sin(t_s \Delta)$$

once, and reuse the corresponding complex multiplier throughout the segment tree traversal. Recomputing trigonometric functions at every node is too expensive.

The number D controls the accuracy-time-memory tradeoff. Small values such as $D = 8, 16, 32$ may already be useful in practice, but the exact choice depends on the value range, the required error tolerance, and the numerical stability requirements.

Since the spectral method is approximate, it is not suitable for traditional online judges requiring exact integer answers without error tolerance. Its main use is as a theoretical approximation framework, a sketching method, or a fast approximate engine.

22 Limitations

The linear exact upper bound assumes bounded machine integers. For arbitrary real or comparison-only values, the direct fallback remains $O(m \log m)$. Even in the integer model, the method uses $O(m)$ temporary workspace and must inspect the query values; it is not a compact dynamic summary.

The $O(n^{4/3} \text{polylog} n)$ pair-space result is theoretical and relies on range-searching structures over $O(n^2)$ pair points. It is best viewed as a compilation tradeoff and is not intended to be practical for large n .

The lazy-incremental transform now has a complete materialized instantiation through the shift-meld histogram segment tree, but this does not produce generic polylogarithmic exact operations. A pull near the root can still cost linear work when both child histograms have large distinct support, and a query with large canonical support falls back to reading all M values. The separate buffered Matrix-CDQ refinement remains conditional on controlling component interactions, query-triggered flushes, and configuration reuse; the shift-meld structure supplies a guaranteed compaction primitive rather than removing these costs.

The Tripartite-3SUM result is a conditional time trade-off, not an unconditional cell-probe lower bound. It proves conditional near-optimality only for the relevant combination of preprocessing and whole-array operation costs; it does not rule out faster structured-instance algorithms or determine the complete preprocessing-update-query frontier.

The approximation guarantees are additive. Exact-far/approximate-near routing reduces the error budget to the pairs actually left in local near-cusp blocks, but if the residual true answer is very small, additive error may still translate into poor relative error. Improving the relative-error behavior requires a more delicate approximation kernel, multiscale normalization, or certified exact treatment of additional residue blocks.

Floating-point numerical error can also accumulate after many updates. In long-running applications, periodic rebuilding or normalization may be necessary.

The kernel construction is also important. The theorem assumes a uniform approximation guarantee on the relevant difference range. In practical implementations, the value range should be estimated or dynamically controlled.

23 Practical Application Scenarios

The problem is not limited to contest-style dynamic arrays. It models settings in which objects are arranged in an ordered domain, each object has a low-dimensional feature vector, and an interval update represents a coherent shift applied to a contiguous time window, geographical strip, or ordered group. A range query then measures the internal dispersion of the selected group by the total pairwise Manhattan distance.

Regional pressure and stress maps. In a map-based monitoring system, each index may represent a cell, station, or ordered regional unit, and the point coordinates may encode pressure, load, temperature, risk level, or other normalized indicators. An update over an interval represents a regional shock or correction, while a query asks how internally heterogeneous a selected region is. The pairwise Manhattan sum is useful here because it aggregates all pairwise disagreements rather than only comparing each point with a centroid.

Sensor and environmental telemetry. For a time-ordered sensor stream, each point can store several measurements such as temperature, humidity, pressure, or

signal strength. Calibration corrections, weather-front effects, and batch sensor adjustments naturally appear as interval translations. A query over a time window then returns a dispersion score measuring how different the measurements inside that window are after all corrections.

Network, system, and security logs. Events in a log stream can be embedded into low-dimensional feature vectors such as latency, volume, error count, or risk score. Interval updates can represent policy changes, normalization shifts, or known measurement bias affecting a contiguous time range. Pairwise Manhattan aggregation provides a window-level heterogeneity measure that can be used as an input to anomaly detection or workload characterization.

Market and operational analytics. In financial or operational time series, each index may represent an asset group, warehouse, route, or time bucket. Interval translations model shocks, price adjustments, demand corrections, or normalization changes. Querying the pairwise Manhattan sum over a range gives a robust dispersion statistic that captures spread across all pairs in the selected group.

Algorithmic benchmarking. The same formulation also gives a compact benchmark for dynamic data structures. It combines range translations, interval queries, non-monoid aggregation, and value-order dependence. This makes it a useful test case for comparing exact, approximate, cache-aware, and fragmentation-aware data-structure techniques.

24 Conclusion

We studied dynamic range pairwise Manhattan aggregation under range translations. The three-dimensional problem decomposes into three one-dimensional dynamic pairwise L_1 aggregation problems, and the one-dimensional problem already contains the essential difficulty: exact answers require information about the value ordering induced inside the query interval after all lazy shifts.

The exact part of the paper now has a matched upper-lower-bound narrative. For arbitrary comparable values, materialization and comparison sorting answer a length- m query in $O(m \log m + \log n)$ time. For fixed-width integers, radix-statistical order recovery improves this to $O(m + \log n)$, with $O(\log n)$ updates and linear persistent space. This is the direct practical endpoint: it removes the sorting logarithm but not the need to process the value distribution.

Pair-space range searching occupies a different point in the tradeoff. It compiles $O(n^2)$ pair-difference breakpoints so that Cross interactions become polylogarithmic, and epoch rebuilding yields $O(n^{4/3} \text{polylog } n)$ amortized operations. The segment-tree interaction spine, fragmentation parameters $g, \eta_Q, \chi_Q, \omega_Q, D_Q$, separation pruning, dirty correction, and memoization are acceleration layers

that exploit favorable workloads and fall back to direct order recovery when the instance is adversarial.

The hierarchical matrix route also admits an exact lazy-incremental interpretation. Value histograms form Laurent polynomials under the monomial action of translation; pair-difference correlations are the discrete curvature of translated cross functions; and threshold features turn partial translations into box convolutions with tent-kernel quadratic corrections. Full-cover updates therefore remain scalar lazy tags, while partial updates produce structured two-term deltas. The shift-meld histogram segment tree makes this representation fully operational: persistent weighted ordered maps support constant-time common shifts, work-optimal support-sensitive pulls, and temporary canonical-map melds for exact queries. Its worst cases remain linear, but the cost is now an explicit function of distinct-value support rather than an unspecified rebuild. Larger symbolic buffers can still be compacted by the rank-structured Matrix-CDQ machinery.

The exact ordering barrier is supported by a direct Tripartite-3SUM reduction. For two fixed blocks, the discrete second difference of

$$H(t) = \sum_{a \in A, b \in B} |a + b - t|$$

recovers exact pair-sum multiplicities. Consequently, under the Tripartite 3SUM Hypothesis, truly subquadratic preprocessing and truly sublinear update-plus-query time cannot coexist, even for positive integers, a fixed updated suffix, and whole-array queries. With light preprocessing and logarithmic updates, the $O(n)$ full-range radix query is therefore conditionally optimal up to $n^{o(1)}$ factors.

The approximate part replaces dynamic ordering by finite-dimensional algebra. In the algebraic CDQ model, a kernel approximation $|x - y| \approx \Phi(x)^T A \Phi(y)$ turns cross interactions into bilinear products between feature sums. Fourier features lead to the Lazy Spectral Segment Tree, where range addition becomes phase multiplication and range queries are computed from spectral sums. Polynomial, Chebyshev, multiscale, and hybrid bases fit the same template and give different approximation and stability tradeoffs.

Taken together, the results form one coherent frontier. Exact fixed-width computation reaches linear query time by radix order recovery; exact value-domain divide and conquer exposes a sparse hierarchical low-rank factorization of the ordered absolute-distance kernel; the curvature and histogram transforms make translations lazy and partial updates algebraically incremental on structured states; the concrete shift-meld structure realizes this algebra with explicit support-sensitive bounds; dirty regrouping removes the quadratic residue repair; kinetic rent-or-buy compilation exploits repeated relative-offset queries; index-boundary entropy controls the clean frontier; the 3SUM reduction makes a strongly sublinear general exact query conditionally implausible under light preprocessing; quadratic pair-space compilation moves cost into preprocessing; and spectral algebra obtains polylogarithmic operations by accepting additive error

only on unresolved near-cusp blocks, while separated far blocks remain exact. Interaction routing remains valuable, but its role is now precise: it detects instances on which the linear exact fallback or the approximate surrogate need not pay their full generic cost.

25 Open Questions

Several natural questions remain open.

- Can the exact integer problem beat linear full-range query time with subquadratic preprocessing under a weaker assumption than refuting Tripartite 3SUM, or is there an unconditional cell-probe barrier?
- Can the Tripartite-3SUM reduction be strengthened to a tight preprocessing–update–query tradeoff, separated into individual lower bounds on U and Q , or extended from amortized to worst-case operation bounds?
- Can the shift–meld update bound be improved below its linear worst case on a broad nontrivial input class, or can one prove a matching lower bound for materialized histogram structures?
- Can the Laurent-histogram and curvature representation support sublinear buffered component-correlation and compaction costs while retaining worst-case query guarantees?
- Can the factorization parameters η_Q and ζ_Q be maintained under run translations without rebuilding the affected value-tree paths, perhaps through a kinetic, carry-aware, or component-shared representation of shifted dyadic supports?
- Can the batch bound based on $\text{MM}(B, g, h_Q)$ yield a nontrivial end-to-end dynamic or offline exponent once factor-construction and configuration-reuse costs are included?
- Can the fragmentation parameters $g, \eta_Q, \zeta_Q, \chi_Q, \omega_Q, D_Q$ be maintained with strong worst-case guarantees rather than serving mainly as an adaptive planning model?
- Can hot-pair compilation be made robust to adversarial version invalidation, for example by persistent subobject reuse or a competitive policy that accounts for rebuild churn?
- Which deterministic kernel families give the best practical tradeoff between additive error, numerical stability, and translation cost for the spectral segment tree?
- Can the exact-far/approximate-near decomposition be strengthened to a relative-error guarantee, perhaps by adaptively refining blocks whose residual contribution is small?

These questions mark the remaining boundary. The principal exact scale is no longer mysterious: radix aggregation gives linear order recovery, while the 3SUM reduction conditionally blocks a strongly sublinear replacement under light preprocessing. What remains open is the precise tradeoff curve, unconditional lower bounds, and the extent to which structured workloads allow the adaptive layer to avoid the generic linear cost.

References

- [1] G. J. Székely, M. L. Rizzo, and N. K. Bakirov. Measuring and testing dependence by correlation of distances. *The Annals of Statistics*, 35(6):2769–2794, 2007.
- [2] G. J. Székely and M. L. Rizzo. Brownian distance covariance. *The Annals of Applied Statistics*, 3(4):1236–1265, 2009.
- [3] A. Rahimi and B. Recht. Random features for large-scale kernel machines. In *Advances in Neural Information Processing Systems*, 2007.
- [4] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars. *Computational Geometry: Algorithms and Applications*. Springer, 3rd edition, 2008.
- [5] L. N. Trefethen. *Approximation Theory and Approximation Practice*. SIAM, 2013.
- [6] J. C. Mason and D. C. Handscomb. *Chebyshev Polynomials*. Chapman and Hall/CRC, 2002.
- [7] G. E. Blelloch, D. Ferizovic, and Y. Sun. Just join for parallel ordered sets. In *Proceedings of the 28th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 253–264, 2016. doi:10.1145/2935764.2935768.
- [8] P. M. Fenwick. A new data structure for cumulative frequency tables. *Software: Practice and Experience*, 24(3):327–336, 1994.
- [9] J. L. Bentley. Multidimensional divide-and-conquer. *Communications of the ACM*, 23(4):214–229, 1980.
- [10] J. L. Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.
- [11] M. Deng, Y. Kirkpatrick, V. Rong, V. Vassilevska Williams, and Z. Zhong. New additive approximations for shortest paths and cycles. In *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022)*, volume 229 of Leibniz International Proceedings in Informatics (LIPIcs), Article 50, pages 50:1–50:10, 2022. doi:10.4230/LIPIcs.ICALP.2022.50.
- [12] W. Hackbusch. *Hierarchical Matrices: Algorithms and Analysis*. Springer Series in Computational Mathematics, volume 49. Springer, 2015. doi:10.1007/978-3-662-47324-5.
- [13] P. K. Agarwal and J. Erickson. Geometric range searching and its relatives. In *Advances in Discrete and Computational Geometry*, pages 1–56, American Mathematical Society, 1999.
- [14] R. Grossi, A. Gupta, and J. S. Vitter. High-order entropy-compressed text indexes. In *Proceedings of the Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 841–850, 2003.
- [15] B. Chazelle. Filtering search: a new approach to query-answering. *SIAM Journal on Computing*, 15(3):703–724, 1986.
- [16] D. E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison–Wesley, 2nd edition, 1998.
- [17] A. Gajentaan and M. H. Overmars. On a class of $O(n^2)$ problems in computational geometry. *Computational Geometry*, 5(3):165–185, 1995.
- [18] M. Pătraşcu. Towards polynomial lower bounds for dynamic problems. In *Proceedings of the 42nd ACM Symposium on Theory of Computing (STOC)*, pages 603–610, 2010.
- [19] T. Kopelowitz, S. Pettie, and E. Porat. Higher lower bounds from the 3SUM conjecture. In *Proceedings of the 27th ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1272–1287, 2016.